

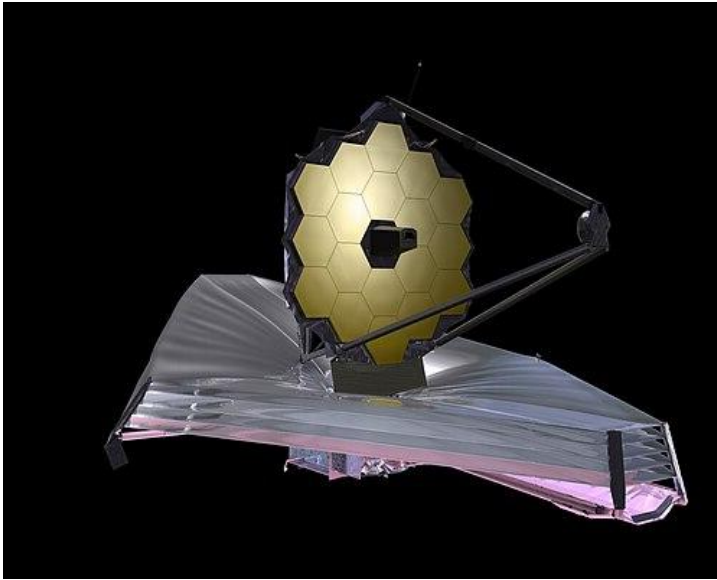
Étude de la dynamique orbitale du James Webb Space Telescope et analyse des corrections nécessaires à son maintien en orbite.

Cycles, boucles

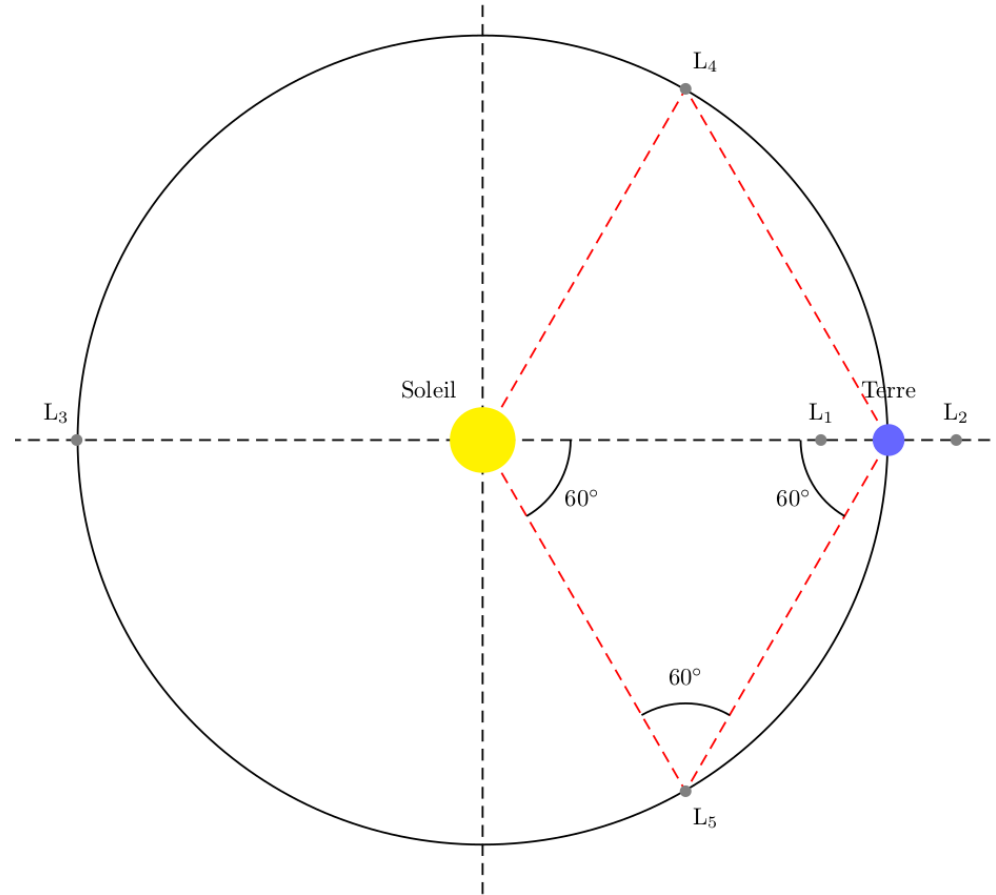
31762 – AVERSENG Victor – MP INFO

Travail de groupe

Le point de Lagrange L2, un positionnement stratégique



wikipedia.fr



wikipedia.fr

Comment ajuster la position du JSWT pour rester en orbite autour du point L2 ?

Sommaire

- Présentation du problème
- Mise en place du modèle physique
- Valeurs Clefs et paramètres de référence
- Première visualisation du mouvement du JWST sans correction
- Méthodes pour la correction

Le modèle CR3BP

(Circular Restricted Three-Body Problem)

- Trois dimensions
- Repère tournant
- Centré sur le barycentre du système
- Axes fixes:
 - Ox: La droite passant par le barycentre et la Terre
 - Oy: Orthogonal à x et sur le plan de l'écliptique
 - Oz: Orthogonal aux deux premiers

Le modèle CR3BP

(Circular Restricted Three-Body Problem)

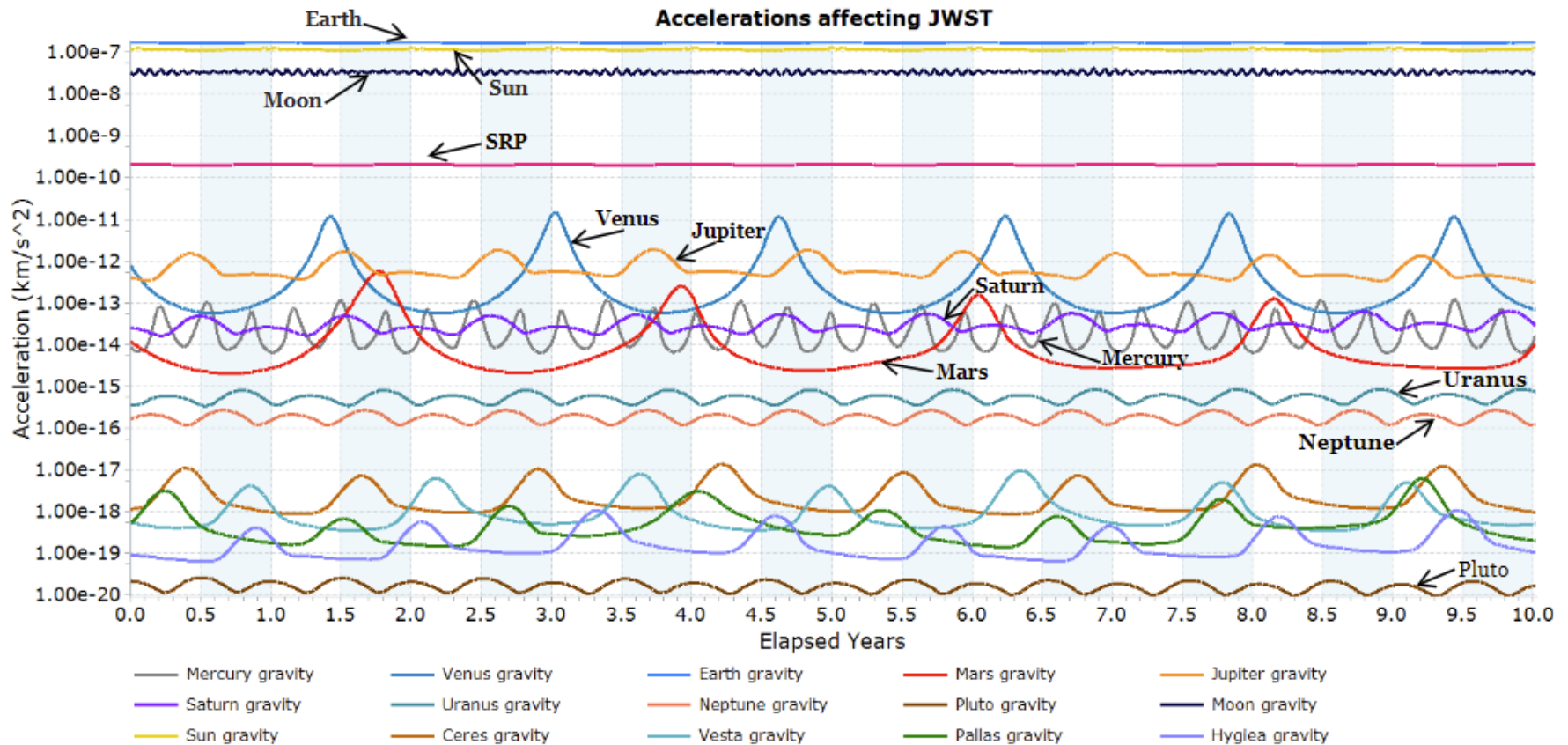
Trois corps principaux:

- Soleil: 2×10^{30} kg
- Terre: 6×10^{24} kg
- JWST: 6×10^3 kg → Sa masse est négligeable

$$\mu = \frac{m_{\text{terre}}}{m_{\text{terre}} + m_{\text{soleil}}} = 3.04 \times 10^{-6}$$

Le modèle CR3BP

(Circular Restricted Three-Body Problem)



[6] James Webb Space Telescope Trajectory Design Overview

Le modèle CR3BP

(Circular Restricted Three-Body Problem)

$$\text{Pos(earth)} = \begin{pmatrix} 1 - \mu \\ 0 \\ 0 \end{pmatrix}$$

$$\text{Pos}(L_2) = \begin{pmatrix} 1 - \mu + \gamma \\ 0 \\ 0 \end{pmatrix}$$

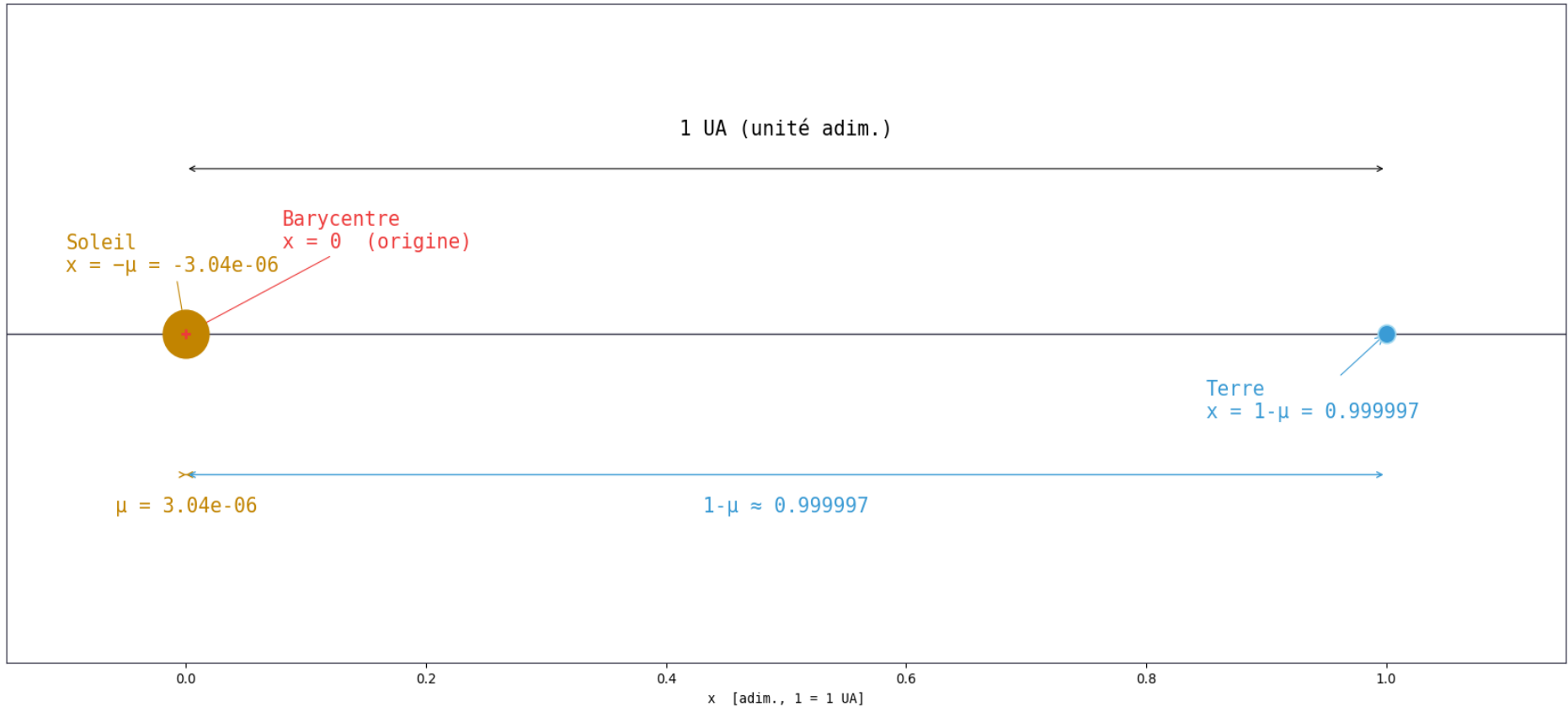
$$\text{Pos(sun)} = \begin{pmatrix} -\mu \\ 0 \\ 0 \end{pmatrix}$$

Où γ est la solution positive de
$$\gamma^5 + (3 - \mu)\gamma^4 + (3 - 2\mu)\gamma^3 - \mu\gamma^2 - 2\mu\gamma - \mu = 0$$

Le modèle CR3BP

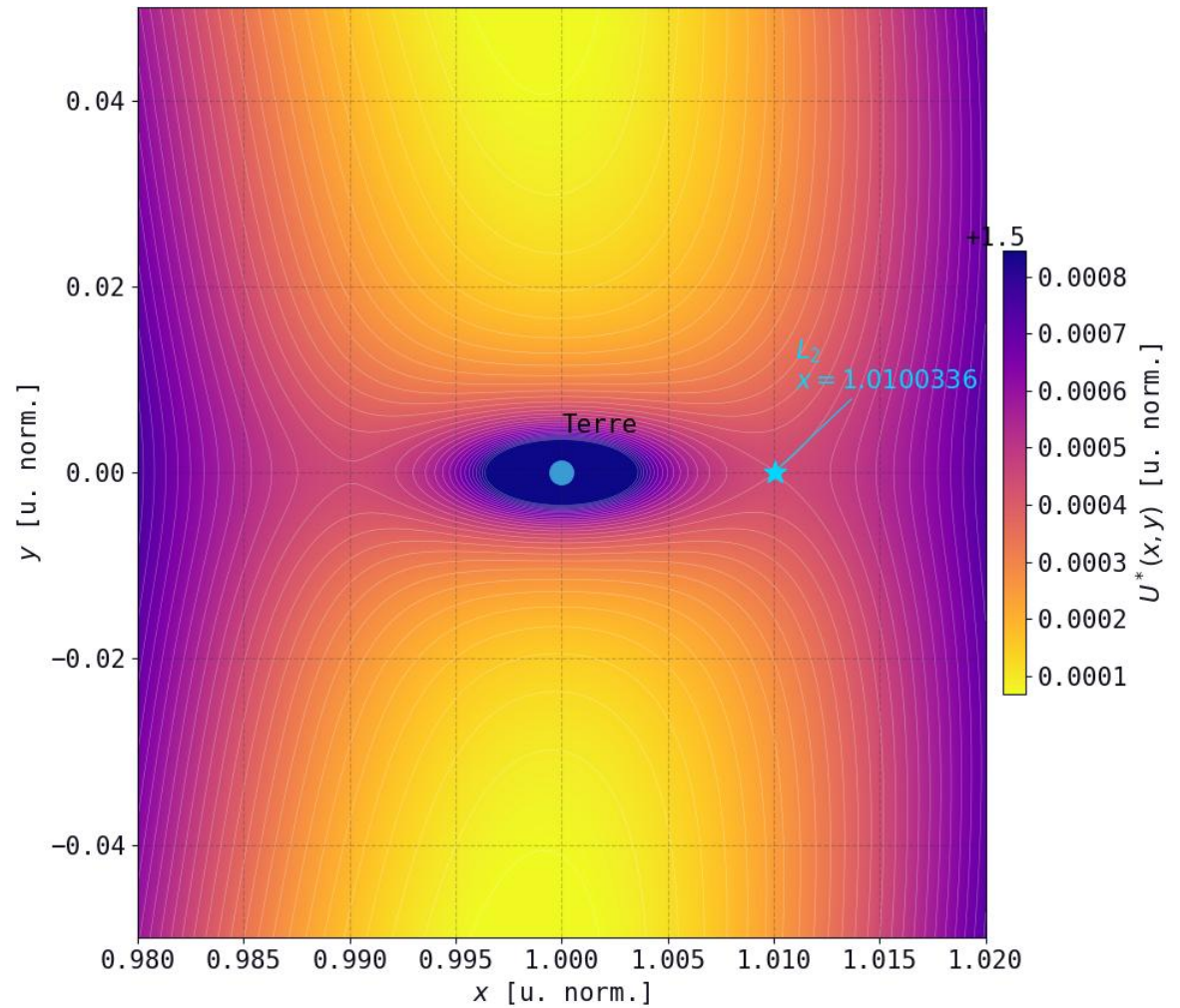
(Circular Restricted Three-Body Problem)

Repère tournant CR3BP – Soleil, Terre, Barycentre
Le barycentre est à 455 000 km du centre du Soleil ($\approx 0.00065 R_{\odot}$)



Potentiel effectif

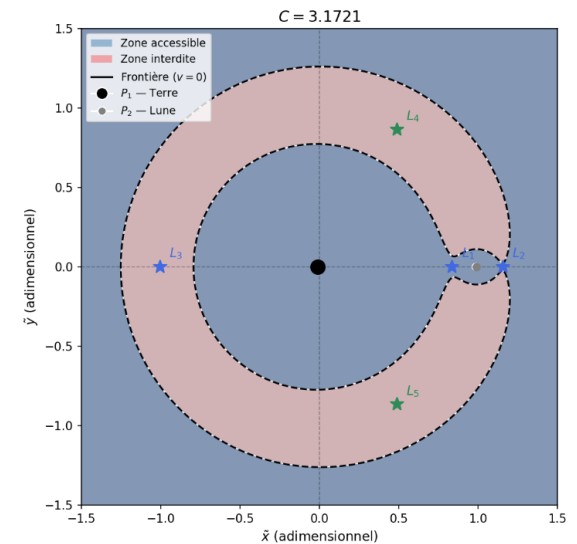
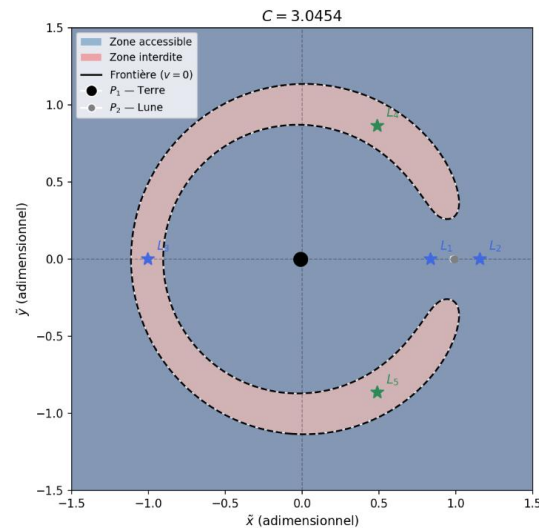
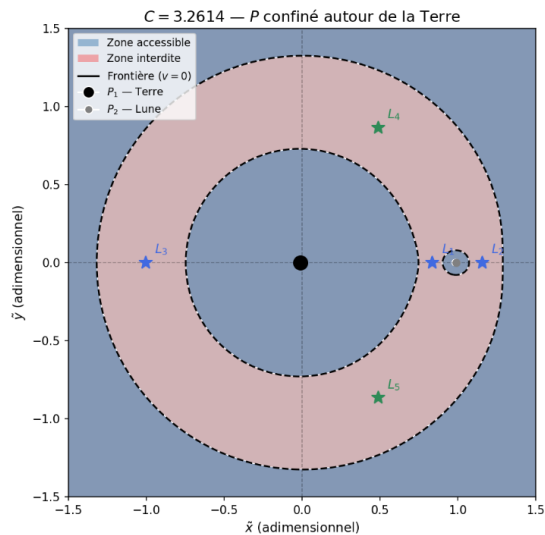
$$U^* = \frac{1 - \mu}{d} + \frac{\mu}{r} + 0.5 \times (x^2 + y^2)$$



La constante de Jacobi C

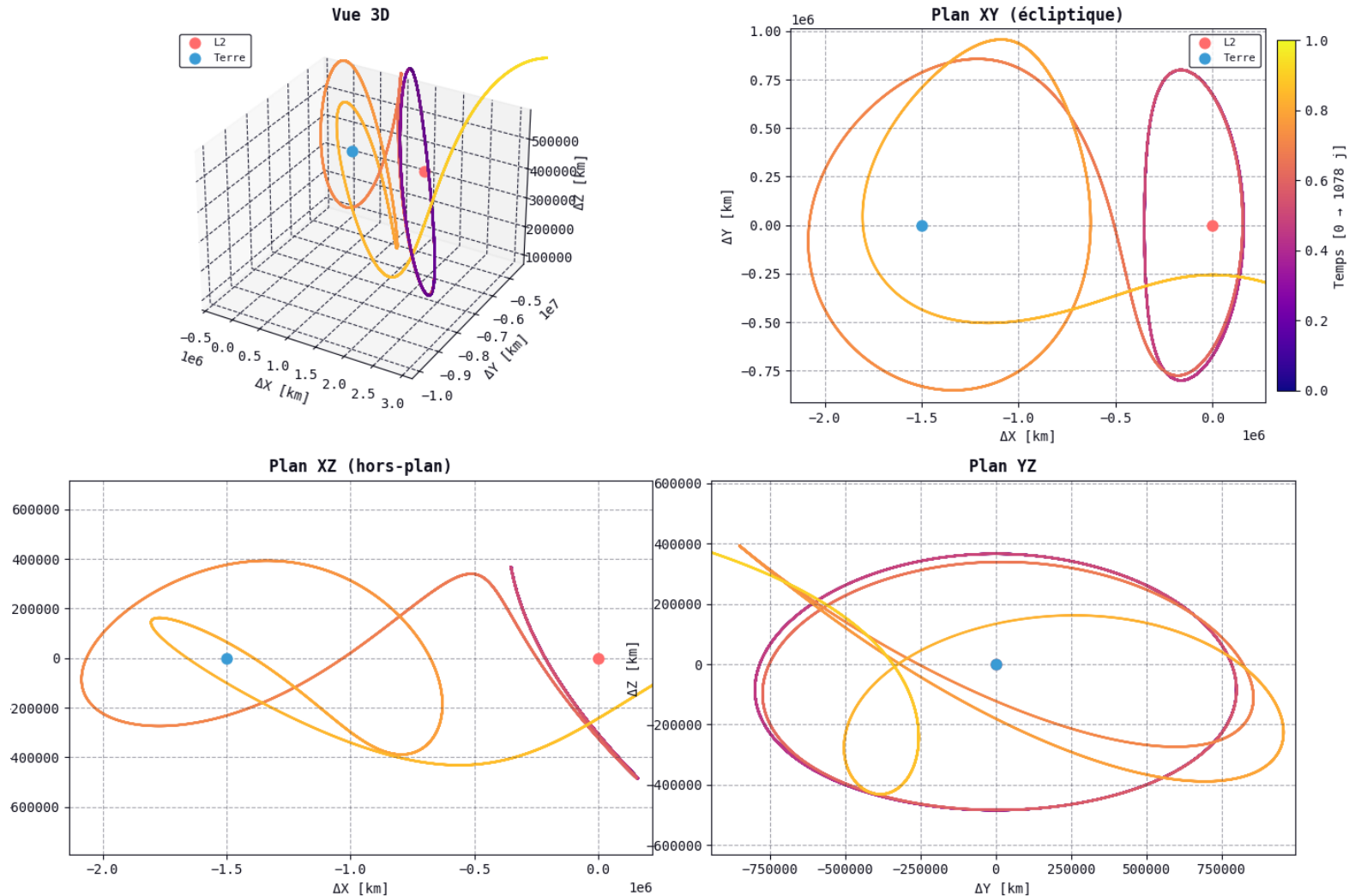
Formule : $C = 2U^* - v^2$ où $v = v_x^2 + v_y^2 + v_z^2$

C'est l'intégrale première du mouvement.



Trajectoire libre du JWST

JWST – Orbite halo autour de L2 (CR3BP, repère tournant)



Intégrateur de Runge-Kutta

On choisit l'ordre 4:

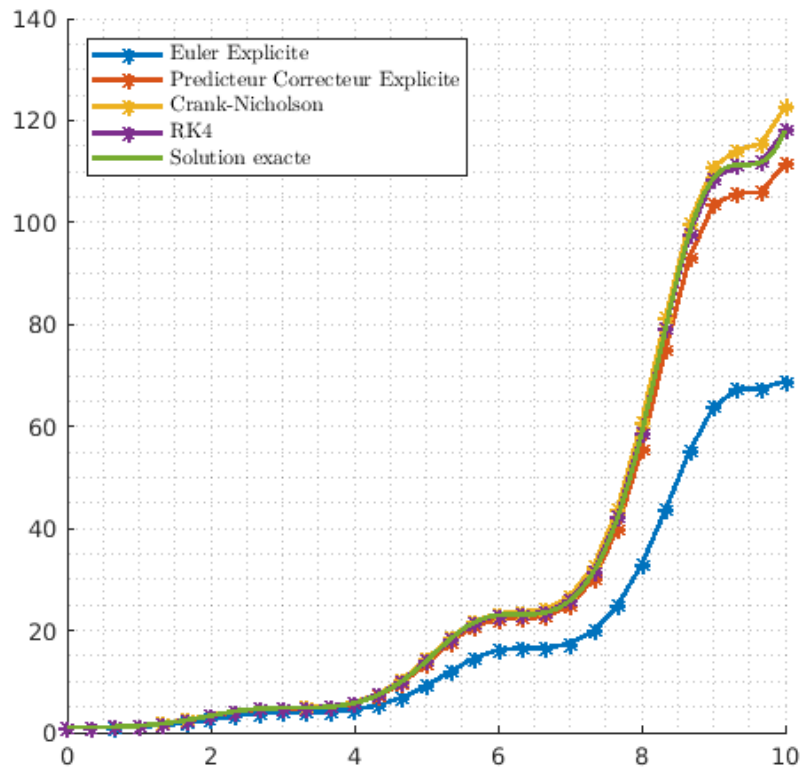
$$\begin{aligned}k_4 &= f(t + h, y_n + h \times k_3) \\k_3 &= f\left(t + \frac{h}{2}, y_n + h \times \frac{k_2}{2}\right) \\k_2 &= f\left(t + \frac{h}{2}, y_n + h \times \frac{k_1}{2}\right) \\k_1 &= f(t, y_n)\end{aligned}$$

$$y_{n+1} = y_n + \frac{h}{6} \times (k_1 + 2 k_2 + 2 k_3 + k_4)$$

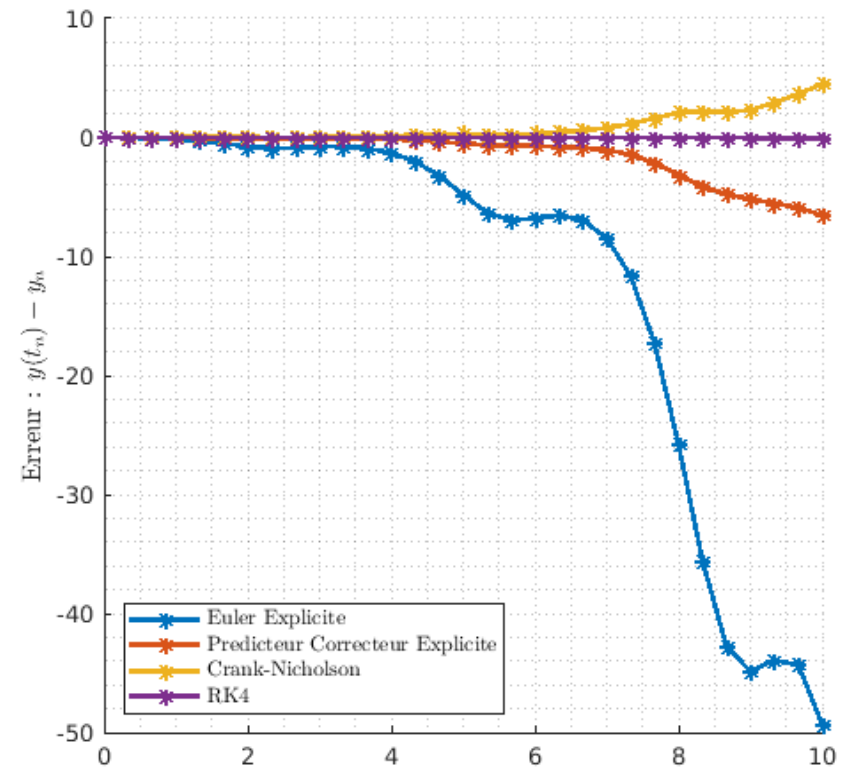
$$\text{Où } h = \frac{T_{halo}}{n_{steps/revolution}} = 3.09 \times 10^{-4} \text{ s}$$

Intégrateur de Runge-Kutta

Exemple avec l'équation: $y' = \sin(t)^2 \times y$

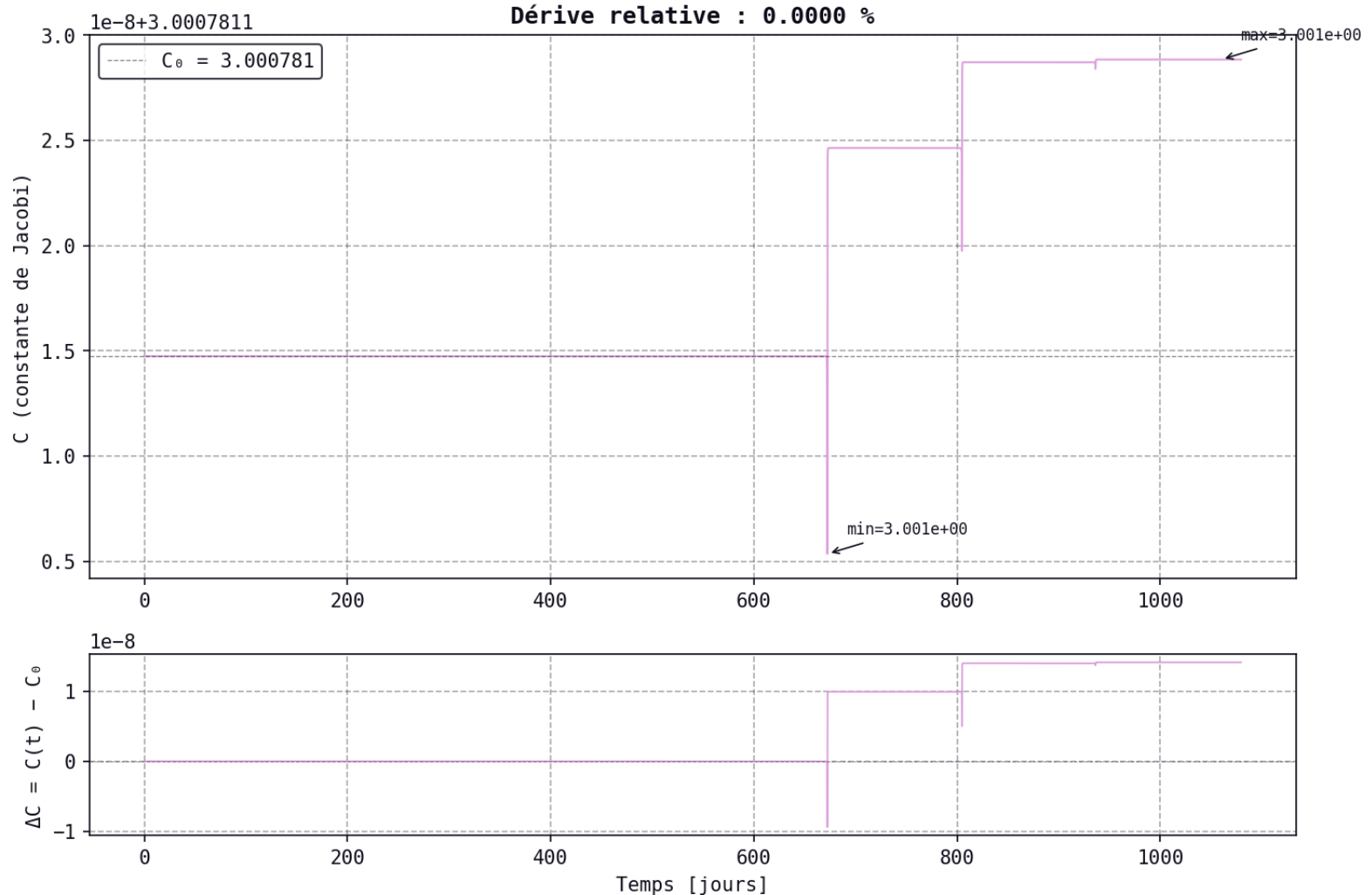


wikipedia.fr



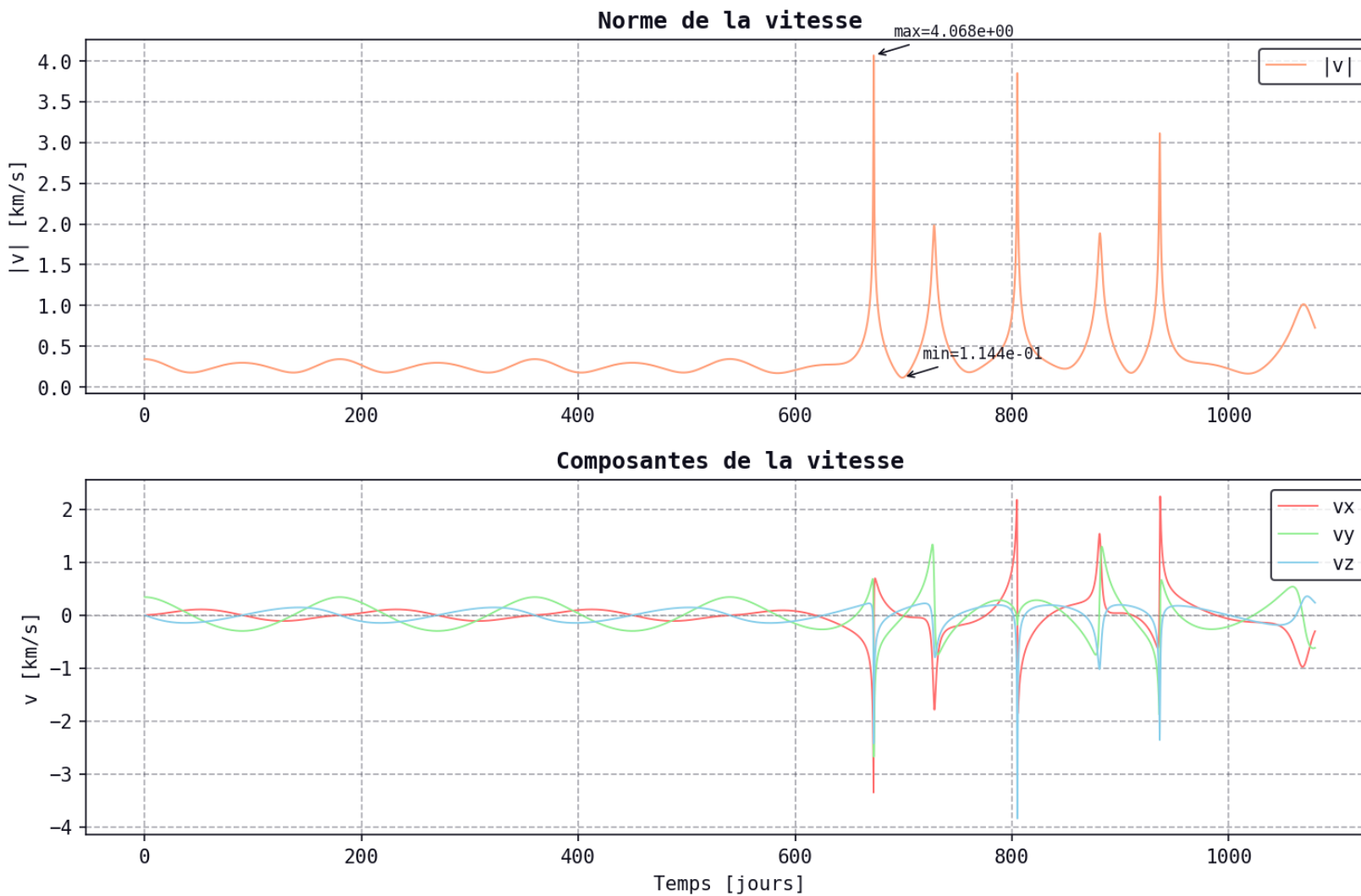
La constante de Jacobi C

CR3BP – Conservation de la constante de Jacobi



Vitesse du JWST

CR3BP – Vitesse du JWST (repère tournant)



Matrice Jacobienne

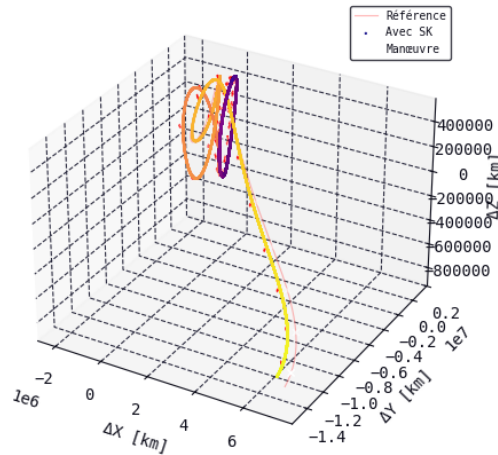
$$A(t) = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ U_{xx}^* & U_{xy}^* & U_{xz}^* & 0 & 2 & 0 \\ U_{yx}^* & U_{yy}^* & U_{yz}^* & -2 & 0 & 0 \\ U_{zx}^* & U_{zy}^* & U_{zz}^* & 0 & 0 & 0 \end{bmatrix}$$

$$U_{ij}^* = \frac{\partial U^*}{\partial i \partial j}$$

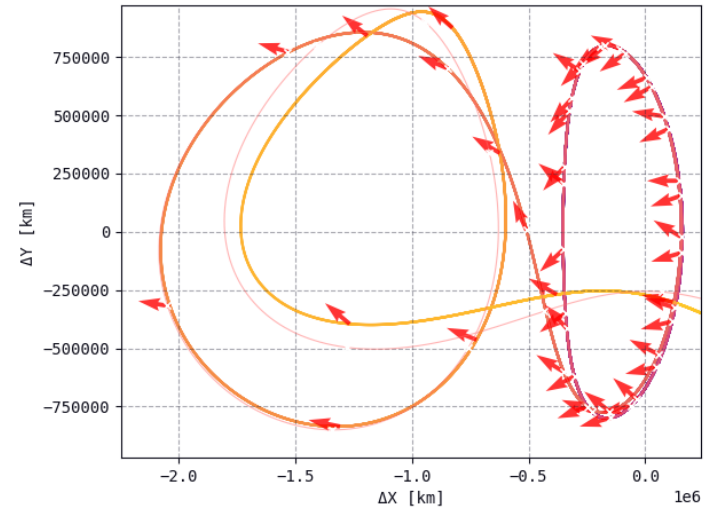
Trajectoire corrigée du JWST

JWST – Station-Keeping EVSK (repère tournant, centré sur L2)

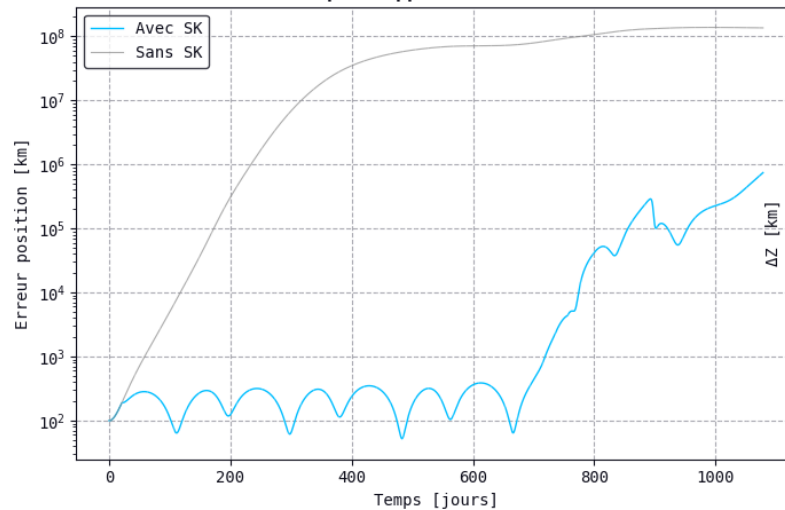
Vue 3D



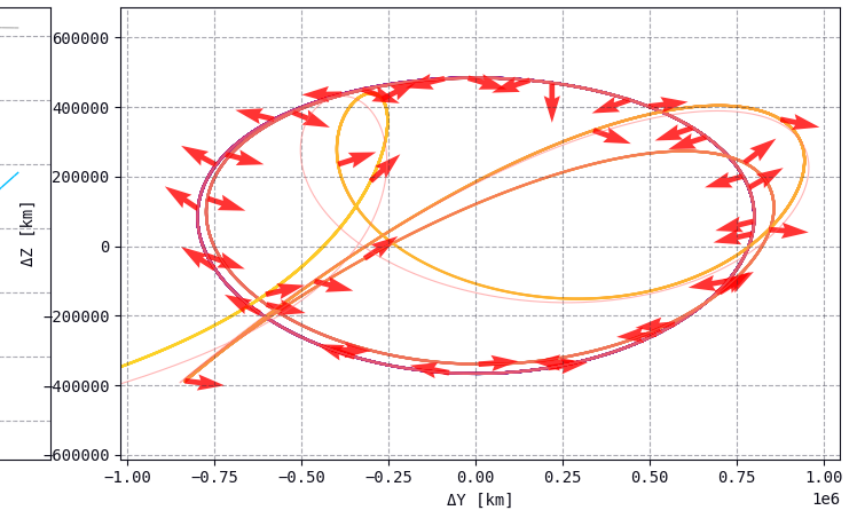
Plan XY



Erreur par rapport à la référence



Plan YZ



Fin

Merci de votre attention !

Annexe – Bibliographie

- [1] WIKIPEDIA CONTRIBUTORS : Point de Lagrange : https://fr.wikipedia.org/wiki/Point_de_Lagrange
- [2] B. MEYSSIGNAC : Introduction à la Mécanique Céleste : https://pagespro.isae-supaero.fr/IMG/pdf/IS101_DRAFT-2.pdf
- [3] J. PETERSEN : L2 station keeping maneuver strategy for the James Webb Space Telescope :
<https://ntrs.nasa.gov/api/citations/20190028877/downloads/20190028877.pdf>
- [4] M. MENZEL, M. DAVIS, K. PARRISH, J. LAWRENCE, A. STEWART, J. COOPER, S. IRISH, G. MOSIER, M. LEVINE, J. PITMAN, G. WALSH, P. MAGHAMI, S. THOMSON, E. WOOLDRIDGE, R. BOUCARUT, L. FEINBERG, ET AL. : The Design, Verification, and Performance of the James Webb Space Telescope :
<https://ntrs.nasa.gov/api/citations/20230009106/downloads/Design%20of%20JWST%20Final%20in%20Publication.pdf>
- [5] WIKIPEDIA CONTRIBUTORS : Runge-Kutta methods : https://en.wikipedia.org/wiki/Runge%E2%80%93Kutta_methods
- [6] W. YU : James Webb Space Telescope Trajectory Design Overview :
https://ntrs.nasa.gov/api/citations/20210017338/downloads/astrocamp_jwst_WayneYu.pdf
- [7] J. BROWN, J. PETERSEN, J. VILLAC, AND W. YU : Seasonal variations of the James Webb Space Telescope orbital dynamics :
<https://ntrs.nasa.gov/api/citations/20150017756/downloads/20150017756.pdf>
- [8] M. CHUPIN : Interplanetary transfers with low consumption using the properties of the restricted three body problem :
Thèse de doctorat, Paris 6, 2016, DOI: 10.70675/4155ee43z02dbz485fzbab9zd36a2807f0b9
- [9] D. L. RICHARDSON : Analytic construction of periodic orbits about the collinear points : Celestial Mechanics 22, (1980), 241-253, DOI: 10.1007/BF01229511

Annexe – main.py

```
1. import numpy as np
2. from cr3bp import CR3BPSimulation
3. from cr3bp.stationkeeping import StationKeepingSimulation
4. from inertial import InertialSimulation
5. from visualization import (
6.     plot_cr3bp_trajectory, plot_cr3bp_velocity,
7.     plot_delta_v_history,
8.     plot_energy,
9.     plot_energy_comparison, plot_inertial_trajectory,
10.    plot_inertial_velocity, plot_jacobi, plot_sk_jacobi,
11.    plot_sk_trajectory, plot_velocity_comparison
12. )
13.
14. # Paramètres communs
15. AZ_ADIM = np.float64(0.00279) # amplitude hors-plan JWST ≈ 418 000 km
16. N_REVOLUTIONS = np.int16(6) # nombre de révolutions halo à simuler
17. N_STEPS = np.int16(10000) # pas RK4 par révolution
18.
19. # 1. Simulation CR3BP
20. print("\n" * 60)
21. print(" MODULE 1 — CR3BP (repère tournant non-dimensionnalisé)")
22. print("\n" * 60)
23.
24. sim_cr3bp = CR3BPSimulation(
25.     Az=AZ_ADIM,
26.     n_revolutions=N_REVOLUTIONS,
27.     n_steps_per_rev=N_STEPS,
28.     northern=True,
29.     phi=np.float64(0),
30. )
31. sim_cr3bp.run()
32.
33. # 2. Station keeping simulation
34. sim_stationkeeping = StationKeepingSimulation(
35.     Az=AZ_ADIM,
36.     n_revolutions=N_REVOLUTIONS,
37.     n_steps_per_rev=N_STEPS,
38. )
39. sim_stationkeeping.run()
40.
41. # 3. Simulation inertielle J2000
42. print("\n" * 60)
43. print(" MODULE 2 — Inertiel (référentiel J2000)")
44. print("\n" * 60)
45.
46. assert sim_cr3bp.state0 is not None and sim_cr3bp.T_halo
47. assert sim_stationkeeping.state0_ref is not None and sim_stationkeeping.T_halo
48.
49. sim_inertial = InertialSimulation(
50.     state0_cr3bp=sim_stationkeeping.state0_ref,
51.     n_revolutions=N_REVOLUTIONS,
52.     T_halo_adim=sim_stationkeeping.T_halo,
53.     n_steps_per_rev=N_STEPS,
54. )
55. sim_inertial.run()
56.
57. # 3. Visualisations
58. print("\nAffichage des graphes...")
59.
60. # Station keeping
61. plot_sk_trajectory(
62.     sim_stationkeeping,
63.     "plots/NoSK_functionnal_Halo/sk_trajectory",
64. )
65. plot_sk_jacobi(
66.     sim_stationkeeping,
67.     "plots/NoSK_functionnal_Halo/sk_jacobi",
68. )
69. plot_delta_v_history(
70.     sim_stationkeeping,
71.     "plots/NoSK_functionnal_Halo/dv_history",
72. )
73.
74. # Trajectoires
75. plot_cr3bp_trajectory(
76.     sim_cr3bp,
77.     "plots/NoSK_functionnal_Halo/cr3bp_trajectory",
78. )
79. plot_inertial_trajectory(
80.     sim_inertial,
81.     "plots/NoSK_functionnal_Halo/inertial_trajectory",
82. )
```

Annexe – main.py

```
81. # Énergie / Jacobi
82. plot_jacobi(
83.     sim_cr3bp,
84.     "plots/NoSK_functionnal_Halo/cr3bp_jacobi",
85. )
86. plot_energy(
87.     sim_inertial,
88.     "plots/NoSK_functionnal_Halo/inertial",
89. )
90. plot_energy_comparison(
91.     sim_cr3bp,
92.     sim_inertial,
93.     "plots/NoSK_functionnal_Halo/energy_comparaison",
94. )
95.
96. # Vitesses
97. plot_cr3bp_velocity(
98.     sim_cr3bp,
99.     "plots/NoSK_functionnal_Halo/cr3bp_velocity",
100. )
101. plot_inertial_velocity(
102.     sim_inertial,
103.     "plots/NoSK_functionnal_Halo/inertial_velocity",
104. )
105. plot_velocity_comparison(
106.     sim_cr3bp,
107.     sim_inertial,
108.     "plots/NoSK_functionnal_Halo/velocity_comparaison",
109. )
```

Annexe – Body.py

```
1. import numpy as np
2. from numpy.typing import NDArray
3.
4. class Body:
5.     """
6.     Corps physique caractérisé par sa masse, sa position et sa vitesse.
7.
8.     Parameters
9.     -----
10.    name : str
11.    mass : np.float64
12.        masse [kg ou adim].
13.    position : NDArray[np.float64], shape (3,)
14.        Vecteur position initial [m ou adim].
15.    velocity : NDArray[np.float64], shape (3,)
16.        Vecteur vitesse initial [m/s ou adim].
17.    fixed : bool
18.        Si True, le corps n'est pas intégré (Soleil, Terre dans le CR3BP).
19.    """
20.
21.    def __init__(
22.        self,
23.        name: str,
24.        mass: np.float64,
25.        position: NDArray[np.float64],
26.        velocity: NDArray[np.float64],
27.        fixed: bool = False,
28.    ):
29.        self.name = name
30.        self.mass = np.float64(mass)
31.        self.position = np.array(position, dtype=np.float64)
32.        self.velocity = np.array(velocity, dtype=np.float64)
33.        self.fixed = fixed
34.
35.        # Historiques accumulés lors de l'intégration
36.        self._pos_history: list[NDArray[np.float64]] = []
37.        self._vel_history: list[NDArray[np.float64]] = []
```

```
38. # État courant
39. @property
40. def state(self) -> NDArray[np.float64]:
41.     """Vecteur d'état [x, y, z, vx, vy, vz]."""
42.     return np.concatenate([self.position, self.velocity], dtype=np.float64)
43.
44. @state.setter
45. def state(self, s: NDArray[np.float64]):
46.     self.position = np.array(s[:3], dtype=np.float64, copy=True)
47.     self.velocity = np.array(s[3:], dtype=np.float64, copy=True)
48.
49. # Gestion de l'historique
50. def record(self):
51.     """Enregistre l'état courant dans l'historique."""
52.     self._pos_history.append(self.position.copy())
53.     self._vel_history.append(self.velocity.copy())
54.
55. def clear_history(self):
56.     self._pos_history.clear()
57.     self._vel_history.clear()
58.
59. @property
60. def pos_history(self) -> NDArray[np.float64]:
61.     """shape (N, 3)"""
62.     return np.array(self._pos_history, np.float64)
63.
64. @property
65. def vel_history(self) -> NDArray[np.float64]:
66.     """shape (N, 3)"""
67.     return np.array(self._vel_history, np.float64)
68.
69. def __repr__(self):
70.     return f"Body('{self.name}', mass={self.mass:.3e}, fixed={self.fixed})"
```

Annexe – integrator.py

```
1. from typing import Callable
2.
3. import numpy as np
4. from numpy.typing import NDArray
5.
6. def rk4_step(
7.     f: Callable[[np.float64, NDArray[np.float64]], NDArray[np.float64]],
8.     t: np.float64,
9.     y: NDArray[np.float64],
10.    h: np.float64,
11. ) -> NDArray[np.float64]:
12.     """
13.     Un pas RK4 classique.
14.
15.     Parameters
16.     -----
17.     f : callable(t, y) -> dy/dt
18.         Fonction dérivée.
19.     t : np.float64
20.         Temps courant.
21.     y : NDArray[np.float64]
22.         État courant.
23.     h : np.float64
24.         Pas de temps.
25.
26.     Returns
27.     -----
28.     y_next : NDArray[np.float64]
29.         État après un pas.
30.     """
31.     k1 = f(t, y)
32.     k2 = f(t + h / 2, y + h * k1 / 2)
33.     k3 = f(t + h / 2, y + h * k2 / 2)
34.     k4 = f(t + h, y + h * k3)
35.
36.     return y + (h / 6) * (k1 + 2 * k2 + 2 * k3 + k4)
37.
38. def integrate(
39.     f: Callable[[np.float64, NDArray[np.float64]], NDArray[np.float64]],
40.     y0: NDArray[np.float64],
41.     t0: np.float64,
42.     t_end: np.float64,
43.     h: np.float64,
44.     callback: Callable[[np.float64, NDArray[np.float64]], None] | None = None,
45. ) -> tuple[NDArray[np.float64], NDArray[np.float64]]:
46.     """
47.     Intégration RK4 sur [t0, t_end].
48.
49.     Parameters
50.     -----
51.     f : callable(t, y) -> dy/dt
52.     y0 : NDArray[np.float64]
53.         État initial.
54.     t0, t_end : np.float64
55.         Bornes de l'intégration.
56.     h : np.float64
57.         Pas de temps fixe.
58.     callback : callable(t, y), optional
59.         Appelé à chaque pas (ex. pour enregistrer des diagnostics).
60.
61.     Returns
62.     -----
63.     t_arr : NDArray[np.float64], shape (N,)
64.     y_arr : NDArray[np.float64], shape (N, len(y0))
65.     """
66.     n_steps = int(np.ceil((t_end - t0) / h))
67.     # Dernier pas potentiellement raccourci
68.     times = np.empty(n_steps + 1, dtype=np.float64)
69.     states = np.empty((n_steps + 1, len(y0)), dtype=np.float64)
70.
71.     t = t0
72.     y = y0.copy()
73.     times[0] = t
74.     states[0] = y
```

Annexe – integrator.py

```
74. if callback is not None:
75.     callback(t, y)
76.
77. for i in range(1, n_steps + 1):
78.     h_eff = min(h, t_end - t) # raccourcit le dernier pas si nécessaire
79.     y = rk4_step(f, t, y, h_eff)
80.     t += h_eff
81.     times[i] = t
82.     states[i] = y
83.
84. if callback is not None:
85.     callback(t, y)
86.
87. return times, states
```

Annexe – system.py

```
1. import numpy as np
2. from numpy.typing import NDArray
3.
4. from .body import Body
5.
6. class System:
7.     """
8.     Ensemble de corps physiques constituant la simulation.
9.
10.    Parameters
11.    -----
12.    bodies : list[Body]
13.        Corps du système (ordre libre, mais conventionnellement :
14.        corps massifs fixes en premier, spacecraft en dernier).
15.    """
16.    def __init__(self, bodies: list[Body]):
17.        self.bodies = bodies
18.        self._time_history: list[np.float64] = []
19.
20.    # Accesseurs
21.    def __getitem__(self, name: str) -> Body:
22.        """Retourne un corps par son nom. Lève ValueError si absent."""
23.        for b in self.bodies:
24.            if b.name == name:
25.                return b
26.        raise ValueError(f"Corps '{name}' introuvable dans le système.")
27.
28.    @property
29.    def massive_bodies(self) -> list[Body]:
30.        """Corps fixes (Soleil, Terre...) — sources de gravité."""
31.        return [b for b in self.bodies if b.fixed]
32.
33.    @property
34.    def free_bodies(self) -> list[Body]:
35.        """Corps intégrés (spacecraft...)."""
36.        return [b for b in self.bodies if not b.fixed]
```

```
37. # Historique temporel
38. def record_time(self, t: np.float64):
39.     t = np.float64(t)
40.     self._time_history.append(t)
41.
42. def clear_history(self):
43.     self._time_history.clear()
44.     for b in self.bodies:
45.         b.clear_history()
46.
47. @property
48. def time_history(self) -> NDArray[np.float64]:
49.     return np.array(self._time_history, dtype=np.float64)
50.
51. # Diagnostics rapides (extrema pour débogage)
52. def print_summary(self, label: str = ""):
53.     """Affiche quelques valeurs extremum après intégration."""
54.     tag = f"[{label}] " if label else ""
55.     print(f"\n{tag}Résumé de simulation")
56.     print(f" Pas de temps enregistrés : {len(self._time_history)}")
57.     for b in self.free_bodies:
58.         if len(b._pos_history) == 0:
59.             continue
60.         pos = b.pos_history
61.         vel = b.vel_history
62.         speeds = np.linalg.norm(vel, axis=1)
63.         print(f" {b.name}:")
64.         print(
65.             f" |r| min/max : {np.linalg.norm(pos, axis=1).min():.4e} / "
66.             f"{np.linalg.norm(pos, axis=1).max():.4e}"
67.         )
68.         print(f" |v| min/max : {speeds.min():.4e} / {speeds.max():.4e}")
69.
70. def __repr__(self):
71.     names = [b.name for b in self.bodies]
72.     return f"System({names})"
```

Annexe – equations.py

```
1. import numpy as np
2. from numpy.typing import NDArray
3.
4. # Paramètre de masse Sun–Earth (valeur JPL)
5. MU_SUN_EARTH = np.float64(3.040423389123456e-6)
6.
7. def distances(
8.     x: np.float64, y: np.float64, z: np.float64, mu: np.float64
9. ) -> tuple[np.float64, np.float64]:
10.     """
11.     Distances adim. du spacecraft au Soleil (d) et à la Terre (r).
12.
13.     d = ||spacecraft - Soleil||, Soleil en (-μ, 0, 0)
14.     r = ||spacecraft - Terre||, Terre en (1-μ, 0, 0)
15.     """
16.     d = np.sqrt((x + mu)**2 + y**2 + z**2, dtype=np.float64)
17.     r = np.sqrt((x - 1 + mu)**2 + y**2 + z**2, dtype=np.float64)
18.     return d, r
19.
20. def pseudo_potential(
21.     x: np.float64, y: np.float64, z: np.float64, mu: np.float64
22. ) -> np.float64:
23.     """Pseudo-potentiel U* (scalaire)."""
24.     d, r = distances(x, y, z, mu)
25.     return (1 - mu) / d + mu / r + 0.5 * (x**2 + y**2)
26.
27. def jacobi_constant(state: NDArray[np.float64], mu: np.float64) -> np.float64:
28.     """
29.     Constante de Jacobi C = 2U* - v².
30.
31.     C est une intégrale première du CR3BP : sa conservation est
32.     l'indicateur principal de la qualité numérique de l'intégration.
33.
34.     Parameters
35.     -----
36.     state : array (6,) [x, y, z, vx, vy, vz] adim.
37.     mu : np.float64
38.     """
39.     x, y, z, vx, vy, vz = state
40.     v2 = vx**2 + vy**2 + vz**2
41.     return 2 * pseudo_potential(x, y, z, mu) - v2
42.
43. def eom(
44.     t: np.float64, state: NDArray[np.float64], mu: np.float64
45. ) -> NDArray[np.float64]:
46.     """
47.     Dérivée du vecteur d'état dans le CR3BP.
48.
49.     Parameters
50.     -----
51.     t : np.float64 temps adim. (non utilisé, système autonome)
52.     state : NDArray [x, y, z, vx, vy, vz]
53.     mu : np.float64
54.
55.     Returns
56.     -----
57.     dstate : NDArray [vx, vy, vz, ax, ay, az]
58.     """
59.     x, y, z, vx, vy, vz = state
60.     d, r = distances(x, y, z, mu)
61.     d3 = d**3
62.     r3 = r**3
63.
64.     # Dérivées partielles de U*
65.     dUx = x - (1 - mu) * (x + mu) / d3 - mu * (x - 1 + mu) / r3
66.     dUy = y * (1 - (1 - mu) / d3 - mu / r3)
67.     dUz = z * (-(1 - mu) / d3 - mu / r3) # terme centrifuge nul en z
68.
69.     # Termes de Coriolis : -2ÿ sur ax, +2x sur ay
70.     ax = dUx + 2 * vy
71.     ay = dUy - 2 * vx
72.     az = dUz
73.
74.     return np.array([vx, vy, vz, ax, ay, az], dtype=np.float64)
75.
76. def eom_factory(mu: np.float64):
77.     """Retourne une fonction eom(t, state) avec mu fixé (pour l'intégrateur)."""
78.
79.     def _eom(t: np.float64, state: NDArray[np.float64]) -> NDArray[np.float64]:
80.         return eom(t, state, mu)
81.     return _eom
```

Annexe – stm.py

```
1. import numpy as np
2. from core.integrator import integrate, rk4_step
3. from numpy.typing import NDArray
4.
5. from .equations import MU_SUN_EARTH, eom
6.
7. ZERO = np.float64(0)
8.
9. # 1. Jacobien analytique des équations CR3BP
10. def jacobian(state: NDArray[np.float64], mu: np.float64) -> NDArray[np.float64]:
11.     """
12.     Jacobien A =  $\partial f / \partial x$  des équations CR3BP, évalué en state.
13.
14.     Structure par blocs :
15.     A = [  $O_3$    $I_3$  ]
16.         [  $\Omega$   C ]
17.
18.      $\Omega$  = matrice des dérivées secondes du pseudo-potentiel U*.
19.     C = [[0, 2, 0], [-2, 0, 0], [0, 0, 0]] (termes de Coriolis).
20.
21.     Parameters
22.     -----
23.     state : NDArray[np.float64], shape (6,) [x, y, z, vx, vy, vz]
24.     mu : np.float64
25.
26.     Returns
27.     -----
28.     A : NDArray[np.float64], shape (6, 6)
29.     """
30.     x, y, z = state[0], state[1], state[2]
31.
32.     d2 = (x + mu) ** 2 + y ** 2 + z ** 2
33.     r2 = (x - 1 + mu) ** 2 + y ** 2 + z ** 2
34.     d3, d5 = d2 ** 1.5, d2 ** 2.5
35.     r3, r5 = r2 ** 1.5, r2 ** 2.5
36.
37.     Uxx = (
38.         1
39.         - (1 - mu) / d3
40.         + 3 * (1 - mu) * (x + mu) ** 2 / d5
41.         - mu / r3
42.         + 3 * mu * (x - 1 + mu) ** 2 / r5
43.     )
44.
45.     Uyy = 1 - (1 - mu) / d3 + 3 * (1 - mu) * y ** 2 / d5 - mu / r3 + 3 * mu * y ** 2 / r5
46.     Uzz = -(1 - mu) / d3 + 3 * (1 - mu) * z ** 2 / d5 - mu / r3 + 3 * mu * z ** 2 / r5
47.     Uxy = 3 * (1 - mu) * (x + mu) * y / d5 + 3 * mu * (x - 1 + mu) * y / r5
48.     Uxz = 3 * (1 - mu) * (x + mu) * z / d5 + 3 * mu * (x - 1 + mu) * z / r5
49.     Uyz = 3 * (1 - mu) * y * z / d5 + 3 * mu * y * z / r5
50.
51.     A = np.zeros((6, 6), dtype=np.float64)
52.     A[:3, :3] = np.eye(3, dtype=np.float64)
53.     A[3:, :3] = np.array(
54.         [[Uxx, Uxy, Uxz], [Uxy, Uyy, Uyz], [Uxz, Uyz, Uzz]], dtype=np.float64
55.     )
56.     A[3:, 3:] = np.array([[0, 2, 0], [-2, 0, 0], [0, 0, 0]], dtype=np.float64)
57.     return A
58.
59. # 2. Équations du système augmenté [état, STM]
60. def eom_stm(
61.     t: np.float64, y: NDArray[np.float64], mu: np.float64
62. ) -> NDArray[np.float64]:
63.     """
64.     Dérivée du vecteur augmenté [x(6), vec( $\Phi$ )(36)].
65.
66.     y[:6] = état CR3BP
67.     y[6:] = STM  $\Phi$  vectorisée colonne par colonne (ordre Fortran)
68.     """
69.     state = y[:6]
70.     Phi = y[6:].reshape(6, 6, order="F")
71.     dstate = eom(t, state, mu)
72.     dPhi = jacobian(state, mu) @ Phi
73.     return np.concatenate([dstate, dPhi.flatten(order="F")], dtype=np.float64)
74.
75. def eom_stm_factory(mu: np.float64):
76.     def _f(t, y):
77.         return eom_stm(t, y, mu)
78.
79.     return _f
80.
81. # 3. Calcul de la matrice de monodromie
82. def compute_monodromy(
83.     state0: NDArray[np.float64],
84.     T: np.float64,
85.     mu: np.float64 = MU_SUN_EARTH,
```

Annexe – stm.py

```
85. n_steps: np.int16 = np.int16(10000),
86. ) -> tuple[NDArray[np.float64], NDArray[np.float64], NDArray[np.float64]]:
87. """
88. Intègre le système augmenté sur une période T.
89.
90. Returns
91. -----
92. M : NDArray[np.float64] (6, 6)  matrice de monodromie  $\Phi(T, 0)$ 
93. times : NDArray[np.float64] (N,)
94. stms : NDArray[np.float64] (N, 6, 6) STM à chaque instant
95. """
96.
97. y0 = np.concatenate([state0, np.eye(6).flatten(order="F")], dtype=np.float64)
98. times, ys = integrate(eom_stm_factory(mu), y0, ZERO, T, T / n_steps)
99. stms = ys[:, 6:].reshape(-1, 6, 6, order="F")
100. return stms[-1], times, stms
101.
102. # 3b. Correction différentielle (tir simple) → orbite halo périodique
103. def _integrate_to_half_period(
104.     state0: NDArray[np.float64],
105.     mu: np.float64,
106.     T_guess: np.float64,
107.     n_steps: int = 20000,
108. ) -> tuple[np.float64, NDArray[np.float64], NDArray[np.float64]]:
109.     """
110.     Intègre le système augmenté [état,  $\Phi$ ] de t=0 jusqu'au PREMIER passage y=0 (t>0).
111.
112.     La graine est sur le plan xz (y0=0) ; le prochain passage y=0 marque la
113.     demi-période. Renvoie (t_cross, état(6) au passage,  $\Phi(6,6)$  au passage).
114.     """
115.     f = eom_stm_factory(mu)
116.     y = np.concatenate([state0, np.eye(6).flatten(order="F")], dtype=np.float64)
117.     h = np.float64(T_guess / n_steps)
118.     t = ZERO
119.     for i in range(n_steps):
120.         y_prev, t_prev = y.copy(), t
121.         y = rk4_step(f, t, y, h)
122.         t = t + h
123.         # passage par zéro de la composante y (indice 1), après avoir quitté y=0
124.         if i > 0 and y_prev[1] * y[1] < 0:
```

```
125.         frac = y_prev[1] / (y_prev[1] - y[1]) # interpolation linéaire du zéro
126.         y_cross = rk4_step(f, t_prev, y_prev, frac * h)
127.         t_cross = t_prev + frac * h
128.         return t_cross, y_cross[6:], y_cross[6:].reshape(6, 6, order="F")
129.     raise RuntimeError("Aucun passage y=0 trouvé dans [0, T_guess].")
130.
131. def correct_halo(
132.     state0: NDArray[np.float64],
133.     mu: np.float64 = MU_SUN_EARTH,
134.     T_guess: np.float64 = np.float64(3.0),
135.     tol: np.float64 = np.float64(1e-11),
136.     max_iter: int = 50,
137. ) -> tuple[NDArray[np.float64], np.float64]:
138.     """
139.     Raffine une graine de Richardson en orbite halo PÉRIODIQUE par tir simple.
140.
141.     On exploite la symétrie des halos par rapport au plan xz : la graine est de la
142.     forme [x0, 0, z0, 0, vy0, 0] (passage perpendiculaire). On intègre jusqu'au
143.     prochain passage y=0 et on impose vx=vz=0 à ce passage, en ajustant (x0, vy0)
144.     à z0 = Az fixé. Newton via la STM, avec correction du temps de vol :
145.
146.         d[vx; vz] = ( [[ $\Phi_{41}$   $\Phi_{45}$ ], [ $\Phi_{61}$   $\Phi_{65}$ ]] - (1/vy_f)[ax_f; az_f]⊗[ $\Phi_{21}$   $\Phi_{25}$ ] ) [dx0; dvy0]
147.
148.     Returns
149.     -----
150.     state0_corr : (6,) condition initiale périodique corrigée
151.     T : période complète vraie (= 2*t_cross)
152.     """
153.     s = np.array(state0, dtype=np.float64).copy()
154.     err = np.float64(np.inf)
155.     for _ in range(max_iter):
156.         t_half, s_c, Phi = _integrate_to_half_period(s, mu, T_guess)
157.         vx_f, vz_f = s_c[3], s_c[5]
158.         err = max(abs(vx_f), abs(vz_f))
159.         if err < tol:
160.             return s, np.float64(2.0 * t_half)
161.         ds = eom(t_half, s_c, mu)
162.         ax_f, az_f = ds[3], ds[5]
163.         vy_f = s_c[4]
164.         Mmat = np.array(
165.             [[Phi[3, 0], Phi[3, 4]], [Phi[5, 0], Phi[5, 4]]], dtype=np.float64
166.         ) - (1.0 / vy_f) * np.outer(
```

Annexe – stm.py

```
167.     np.array([ax_f, az_f]), np.array([Phi[1, 0], Phi[1, 4]])
168. )
169. d = np.linalg.solve(Mmat, -np.array([vx_f, vz_f], dtype=np.float64))
170. s[0] += d[0]
171. s[4] += d[1]
172. T_guess = np.float64(2.0 * t_half)
173. raise RuntimeError(f"Correction différentielle non convergée (err={err:.2e}).")
174.
175. # 4. Extraction des directions stable et instable
176. def stable_unstable_eigvecs(
177.     M: NDArray[np.float64],
178. ) -> tuple[
179.     NDArray[np.float64],
180.     NDArray[np.float64],
181.     NDArray[np.float64],
182.     NDArray[np.float64],
183.     np.complex128,
184.     np.complex128,
185. ]:
186.     """
187.     Extrait les vecteurs propres stable et instable de la monodromie.
188.
189.     Appariement gauche/droit :
190.     Les vecteurs propres gauches (adjoints) sont les vecteurs propres droits
191.     de  $M^T$ . On les apparie avec les valeurs propres de  $M$  par distance minimale
192.     dans le plan complexe, puis on normalise biorthogonalement :
193.      $v_{u\_left} \leftarrow v_{u\_left} / (v_{u\_left} \cdot v_u)$ 
194.     de sorte que  $\langle v_{u\_left}, v_u \rangle = 1$ .
195.
196.     Returns
197.     -----
198.     v_s, v_u, v_s_left, v_u_left : NDArray(6,)
199.     lam_s, lam_u : complex
200.     """
201.     eigvals_R, V_R = np.linalg.eig(M) # droits :  $M v = \lambda v$ 
202.     eigvals_L, V_L = np.linalg.eig(M.T) # gauches :  $M^T w = \lambda w$ 
203.
204.     mods = np.abs(eigvals_R, dtype=np.float64)
```

```
168. # Valeur propre la plus petite en module → stable
169. # Valeur propre la plus grande en module → instable
170. # On exclut les paires complexes en cherchant parmi les réelles
171. real_mask = np.abs(eigvals_R.imag, dtype=np.float64) < 1e-6 * np.abs(
172.     eigvals_R.real + 1e-30, dtype=np.float64
173. ) #
174.
175. if real_mask.sum() >= 2:
176.     real_idx = np.where(real_mask)[0]
177.     idx_s = real_idx[np.argmin(mods[real_idx])]
178.     idx_u = real_idx[np.argmax(mods[real_idx])]
179. else:
180.     idx_s = int(np.argmin(mods))
181.     idx_u = int(np.argmax(mods))
182.
183. lam_s = eigvals_R[idx_s]
184. lam_u = eigvals_R[idx_u]
185.
186. # Appariement des vecteurs gauches : trouver dans eigvals_L celui le plus
187. # proche de lam_s et lam_u respectivement.
188. idx_s_L = int(np.argmin(np.abs(eigvals_L - lam_s)))
189. idx_u_L = int(np.argmin(np.abs(eigvals_L - lam_u)))
190.
191. v_s_vec = V_R[:, idx_s].real
192. v_u_vec = V_R[:, idx_u].real
193. v_s_left_vec = V_L[:, idx_s_L].real
194. v_u_left_vec = V_L[:, idx_u_L].real
195.
196. # Normalisation L2
197. v_s_vec /= np.linalg.norm(v_s_vec)
198. v_u_vec /= np.linalg.norm(v_u_vec)
199.
200. # Normalisation biorthogonale :  $\langle v_{u\_left}, v_u \rangle = 1$ 
201. dot_u = np.dot(v_u_left_vec, v_u_vec)
202. if abs(dot_u) > 1e-14:
203.     v_u_left_vec /= dot_u
204. else:
205.     v_u_left_vec /= np.linalg.norm(v_u_left_vec)
```

Annexe – stm.py

```
206. # Normalisation biorthogonale :  $\langle v_u\_left, v_u \rangle = 1$ 
207. dot_u = np.dot(v_u_left_vec, v_u_vec)
208. if abs(dot_u) > 1e-14:
209.     v_u_left_vec /= dot_u
210. else:
211.     v_u_left_vec /= np.linalg.norm(v_u_left_vec)
212.
213. dot_s = np.dot(v_s_left_vec, v_s_vec)
214. if abs(dot_s) > 1e-14:
215.     v_s_left_vec /= dot_s
216. else:
217.     v_s_left_vec /= np.linalg.norm(v_s_left_vec)
218.
219. return v_s_vec, v_u_vec, v_s_left_vec, v_u_left_vec, lam_s, lam_u
220.
221. def print_monodromy_summary(M: NDArray[np.float64], mu: np.float64 =
    MU_SUN_EARTH):
222.     eigvals = np.linalg.eigvals(M)
223.     mods = np.abs(eigvals)
224.     v_s, _, _, lam_s, lam_u = stable_unstable_eigvecs(M)
225.
226.     print(v_s)
227.     print("\n[Monodromie]")
228.     print(f" Valeurs propres  $|\lambda|$  : {sorted(mods.tolist())}")
229.     print(f"  $\lambda_{stable}$  = {lam_s:.6f} ( $|\lambda_s| = \{abs(lam_s):.4e\}$ ")
230.     print(f"  $\lambda_{instable}$  = {lam_u:.6f} ( $|\lambda_u| = \{abs(lam_u):.4e\}$ ")
231.     print(f" Produit  $|\lambda_s| \cdot |\lambda_u| = \{abs(lam_s)*abs(lam_u):.6f\}$  (attendu  $\approx 1$ )")
```

Annexe – lagrange.py

```
1. import numpy as np
2. from numpy.typing import NDArray
3. from scipy.optimize import brentq
4.
5. from .equations import MU_SUN_EARTH
6.
7. ZERO = np.float64(0)
8. # 1. Points de Lagrange colinéaires (L1, L2, L3)
9. def _gamma_L2(mu: np.float64) -> np.float64:
10.     """
11.     Dimensionless distance between the secondary body and L2
12.     in the circular restricted three-body problem.
13.     """
14.     mu = np.float64(mu)
15.
16.     def eq(g):
17.         return (
18.             g**5 + (3 - mu) * g**4 + (3 - 2 * mu) * g**3 - mu * g**2 - 2 * mu * g - mu
19.         )
20.
21.     g0 = (mu / 3) ** (1 / 3)
22.     return np.float64(brentq(eq, g0 * 0.5, g0 * 1.5))
23.
24. def lagrange_L2(mu: np.float64 = MU_SUN_EARTH) -> NDArray[np.float64]:
25.     """
26.     Position de L2 dans le repère tournant, sous forme [x_L2, 0, 0].
27.     x_L2 = 1 -  $\mu$  +  $\gamma_2$  (légèrement au-delà de la Terre)
28.     """
29.     gamma = _gamma_L2(mu)
30.     x_L2 = 1.0 - mu + gamma
31.     return np.array([x_L2, 0.0, 0.0], dtype=np.float64)
32.
33. # 2. Approximation de Richardson (3ème ordre) pour orbite halo autour de L2
34. def richardson_halo_L2(
35.     Az: np.float64,
36.     mu: np.float64 = MU_SUN_EARTH,
37.     northern: bool = True,
38.     phi: np.float64 = ZERO,
39. ) -> tuple[NDArray[np.float64], np.float64, np.float64]:
40.     """
41.     État initial d'une orbite halo autour de L2, approximation de Richardson.
42.
43.     Parameters
44.     -----
45.     Az : np.float64
46.         Amplitude hors-plan (adim.). Ex : 0.00279 pour JWST.
47.     mu : np.float64
48.         Paramètre de masse du CR3BP.
49.     northern : bool
50.         True → halo nord (z > 0 initialement, m = +1)
51.         False → halo sud (z < 0 initialement, m = -1)
52.     phi : np.float64
53.         Phase initiale sur l'orbite [rad]. phi=0 → départ au plan z=max.
54.
55.     Returns
56.     -----
57.     state : NDArray[np.float64], shape (6,)
58.         [x, y, z, vx, vy, vz] dans le repère tournant adim.
59.     T_half : np.float64
60.         Demi-période approximative de l'orbite halo [adim.].
61.     """
62.     m = 1 if northern else -1
63.     gamma = _gamma_L2(mu)
64.
65.     # Az est fourni en unités CR3BP (barycentriques). Le développement de Richardson
66.     # travaille en coordonnées locales normalisées par  $\gamma$  (distance L2–Terre).
67.     Az = np.float64(Az) / gamma
68.     c = _cn_coefficients(gamma, mu)
69.
70.     c2, c3, c4 = c[2], c[3], c[4]
71.
72.     # Fréquences et coefficients
73.     # _eigenvalues renvoie (taux hyperbolique, fréquence d'oscillation in-plane, hors-plan).
74.     lam_hyp, omega_p, omega_v = _eigenvalues(c2)
75.     lam = omega_p
76.
77.     # Coefficients k, d1, d2
78.     k = (1 + 2 * c2 + lam**2) / (2 * lam)
```

Annexe – lagrange.py

```
77. d1 = 3 * (lam**2) * (k * (6 * (lam**2) - 1) - 2 * lam) / k
78. d2 = 8 * (lam**2) * (k * (11 * (lam**2) - 1) - 2 * lam) / k
79.
80. # Amplitude Ax en fonction de Az (relation de bifurcation halo)
81. # a21...a24, b21...b22 de Richardson
82. a21 = (3 * c3 * (k**2 - 2)) / (4 * (1 + 2 * c2))
83. a22 = (3 * c3) / (4 * (1 + 2 * c2))
84. a23 = -3 * c3 * lam * (3 * lam * (k**3) - 6 * k * (k - lam) + 4) / (4 * k * d1)
85. a24 = -3 * c3 * lam * (2 + 3 * k * lam) / (4 * k * d1)
86.
87. b21 = -3 * c3 * lam * (3 * k * lam - 4) / (2 * d1)
88. b22 = 3 * c3 * lam / d1
89.
90. d21 = -c3 / (2 * lam**2)
91.
92. a31 = -9 * lam * (4 * c3 * (k * a23 - b21) + k * c4 * (4 + k**2)) / (4 * d2) + (
93.     9 * lam**2 + 1 - c2
94. ) * (3 * c3 * (2 * a23 - k * b21) + c4 * (2 + 3 * k**2)) / (2 * d2)
95. a32 = -(
96.     (
97.         9 * lam * (4 * c3 * (k * a24 - b22) + k * c4) / 4
98.         + 3 * (9 * lam**2 + 1 - c2) * (c3 * (k * b22 + d21 - 2 * a24) - c4) / 2
99.     )
100.    / d2
101. )
102.
103. b31 = (
104.     3
105.     * (
106.         8 * lam * (3 * c3 * (k * b21 - 2 * a23) - c4 * (2 + 3 * k**2))
107.         + (1 + 2 * c2 + 9 * lam**2)
108.         * (4 * c3 * (k * a23 - b21) + k * c4 * (4 + k**2))
109.     )
110.    / (8 * d2)
111. )
112. b32 = (
113.     9 * lam * (3 * c3 * (k * b22 + d21 - 2 * a24) - c4)
114.     + (3 / 8) * (9 * lam**2 + 1 + 2 * c2) * (4 * c3 * (k * a24 - b22) + k * c4)
115. ) / d2
116.
117. d31 = (3 / (64 * lam**2)) * (4 * c3 * a24 + c4)
118. d32 = (3 / (64 * lam**2)) * (4 * c3 * a23 - d21 + c4 * (k**2 + 4))

119. temp_denom = 2 * lam * (lam * (1 + k**2) - 2 * k)
120. s1 = (
121.     3 * c3 * (2 * a21 * (k**2 - 2) - a23 * (k**2 + 2) - 2 * k * b21) / 2
122.     - 3 * c4 * (3 * k**4 - 8 * k**2 + 8) / 8
123. ) / temp_denom
124. s2 = (
125.     3 * c3 * (2 * a22 * (k**2 - 2) - a24 * (k**2 + 2) + 2 * k * b22 + 5 * d21) / 2
126.     + 3 * c4 * (12 - k**2) / 8
127. ) / temp_denom
128.
129. a1 = -3 * c3 * (2 * a21 + a23 + 5 * d21) / 2 - 3 * c4 * (12 - k**2) / 8
130. a2 = 3 * c3 * (a24 - 2 * a22) / 2 + 9 * c4 / 8
131.
132. l1 = a1 + 2 * lam**2 * s1
133. l2 = a2 + 2 * lam**2 * s2
134.
135. # Initialisation empirique pour la famille de Halo Sun–Earth de L2
136. # Fréquence corrigée au 3ème ordre
137. omega1 = ZERO # correction 1er ordre nulle pour halo
138.
139. delta = omega_p**2 - omega_v**2
140. Ax = np.sqrt(-(delta + Az**2 * l2) / l1, dtype=np.float64)
141.
142. nu = 1 + s1 * Ax**2 + s2 * Az**2
143.
144. T_half = np.float64(np.pi / (omega_p * nu))
145.
146. tau = ZERO
147. tau1 = omega_p * tau + phi
148.
149. x_L2 = np.float64(lagrange_L2[mu][0])
150.
151. # Développement 3ème ordre de Richardson (coordonnées locales ξ, η, ζ)
152. xi = (
153.     a21 * Ax**2
154.     + a22 * Az**2
155.     - Ax * np.cos(tau1, dtype=np.float64)
156.     + (a23 * Ax**2 - a24 * Az**2) * np.cos(2 * tau1, dtype=np.float64)
157.     + (a31 * Ax**3 - a32 * Ax * Az**2) * np.cos(3 * tau1, dtype=np.float64)
158. )
```

Annexe – lagrange.py

```
159. eta = (  
160.     k * Ax * np.sin(tau1, dtype=np.float64)  
161.     + (b21 * Ax**2 - b22 * Az**2) * np.sin(2 * tau1, dtype=np.float64)  
162.     + (b31 * Ax**3 - b32 * Ax * Az**2) * np.sin(3 * tau1, dtype=np.float64)  
163. )  
164.  
165. zeta = m * (  
166.     Az * np.cos(tau1, dtype=np.float64)  
167.     + d21 * Ax * Az * (np.cos(2 * tau1, dtype=np.float64) - 3)  
168.     + (d32 * Ax**2 * Az - d31 * Az**3) * np.cos(3 * tau1, dtype=np.float64)  
169. )  
170.  
171. # Vitesses (dérivées par rapport à  $\tau = v \cdot t$ , donc  $d\tau/dt = v$ )  
172. xi_dot = (  
173.     omega_p  
174.     * nu  
175.     * (  
176.         + Ax * np.sin(tau1, dtype=np.float64)  
177.         - 2 * (a23 * Ax**2 - a24 * Az**2) * np.sin(2 * tau1, dtype=np.float64)  
178.         - 3 * (a31 * Ax**3 - a32 * Ax * Az**2) * np.sin(3 * tau1, dtype=np.float64)  
179.     )  
180. )  
181.  
182. eta_dot = (  
183.     omega_p  
184.     * nu  
185.     * (  
186.         k * Ax * np.cos(tau1, dtype=np.float64)  
187.         + 2 * (b21 * Ax**2 - b22 * Az**2) * np.cos(2 * tau1, dtype=np.float64)  
188.         + 3 * (b31 * Ax**3 - b32 * Ax * Az**2) * np.cos(3 * tau1, dtype=np.float64)  
189.     )  
190. )  
191.  
192. zeta_dot = (  
193.     nu  
194.     * omega_p  
195.     * m  
196.     * (  
197.         - Az * np.sin(tau1, dtype=np.float64)  
198.         - 2 * d21 * Ax * Az * np.sin(2 * tau1, dtype=np.float64)  
199.         - 3 * (d32 * Ax**2 * Az - d31 * Az**3) * np.sin(3 * tau1, dtype=np.float64)  
200.     )  
201. )
```

```
202. # Passage des coordonnées locales de Richardson (échelle  $\gamma$ ) aux coordonnées  
203. x = x_L2 + gamma * xi  
204. y = gamma * eta  
205. z = gamma * zeta  
206. vx = gamma * xi_dot  
207. vy = gamma * eta_dot  
208. vz = gamma * zeta_dot  
209.  
210. return np.array([x, y, z, vx, vy, vz], dtype=np.float64), T_half, c2  
211.  
212. # Fonctions auxiliaires internes  
213. def _cn_coefficients(  
214.     gamma: np.float64, mu: np.float64, n_max: int = 5  
215. ) -> dict[int, np.float64]:  
216.     """  
217.     Coefficients  $c_n$  du développement du potentiel autour de L2  
218.      $c_n = ((-1)^n / \gamma^3) [ \mu + (1-\mu) \gamma^{(n+1)} / (1+\gamma)^{(n+1)} ]$   
219.  
220.     Soleil-Terre L2 :  $c_2 \approx 3.94$ ,  $c_3 \approx -2.98$ ,  $c_4 \approx 2.97$ .  
221.     """  
222.     c: dict[int, np.float64] = {}  
223.     for n in range(2, n_max + 1):  
224.         c[n] = np.float64(  
225.             ((-1) ** n / gamma**3)  
226.             * (mu + (1 - mu) * gamma ** (n + 1) / (1 + gamma) ** (n + 1))  
227.         )  
228.     return c  
229.  
230. def _eigenvalues(c2: np.float64) -> tuple[np.float64, np.float64, np.float64]:  
231.     """  
232.     Valeurs propres du système  
233.     """  
234.     disc = 9 * c2**2 - 8 * c2  
235.     lam2 = (c2 - 2 + np.sqrt(disc, dtype=np.float64)) / 2  
236.     omega_p2 = (c2 - 2 - np.sqrt(disc, dtype=np.float64)) / 2  
237.     return (  
238.         np.sqrt(lam2, dtype=np.float64),  
239.         np.sqrt(-omega_p2, dtype=np.float64),  
240.         np.sqrt(c2, dtype=np.float64),  
241.     )
```

Annexe – cr3bp/simulation.py

```
1. import numpy as np
2. from core.body import Body
3. from core.integrator import integrate
4. from numpy.typing import NDArray
5.
6. from .equations import MU_SUN_EARTH, eom_factory, jacobi_constant
7. from .lagrange import lagrange_L2, richardson_halo_L2
8. from .stm import correct_halo
9.
10. # Facteur de conversion unité de temps adim → secondes
11. #  $t^* = \sqrt{l^3 / (G m^*)}$  avec  $l^* = 1 \text{ UA}$ ,  $G m^* \approx G(M_{\text{sun}} + M_{\text{earth}}) \approx G M_{\text{sun}}$ 
12. #  $T_{\text{orb\_Terre}} = 2\pi t^* \rightarrow t^* = T_{\text{orb}} / 2\pi = 365.25 * 86400 / 2\pi$ 
13. T_STAR_SEC = np.float64(365.25 * 86400 / (2 * np.pi)) #  $\approx 5.023\text{e}6 \text{ s} \approx 58.15 \text{ jours}$ 
14. ZERO = np.float64(0)
15.
16. class CR3BPSimulation:
17.     """
18.     Simulation du JWST dans le repère tournant du CR3BP.
19.
20.     Parameters
21.     -----
22.     Az : np.float64
23.         Amplitude hors-plan de l'orbite halo [adim.].
24.     mu : np.float64
25.         Paramètre de masse.
26.     n_revolutions : np.int16
27.         Nombre de révolutions halo à simuler.
28.     n_steps_per_rev : np.int16
29.         Nombre de pas RK4 par révolution (qualité de l'intégration).
30.     northern : bool
31.         Halo nord (True) ou sud (False).
32.     phi : np.float64
33.         Phase initiale [rad].
34.     """
35.
36.     def __init__(
37.         self,
38.         Az: np.float64 = np.float64(0.00279),
39.         mu: np.float64 = MU_SUN_EARTH,
40.         n_revolutions: np.int16 = np.int16(4),
41.         n_steps_per_rev: np.int16 = np.int16(5000),
42.         northern: bool = False,
43.         phi: np.float64 = ZERO,
44.     ):
45.         self.Az = Az
46.         self.mu = mu
47.         self.n_revolutions = n_revolutions
48.         self.n_steps_per_rev = n_steps_per_rev
49.         self.northern = northern
50.         self.phi = phi
51.
52.         # Résultats (remplis après run())
53.         self.times: NDArray[np.float64] | None = None # adim.
54.         self.states: NDArray[np.float64] | None = None # shape (N, 6)
55.         self.jacobi: NDArray[np.float64] | None = None # shape (N,)
56.         self.T_halo: np.float64 | None = None # période adim.
57.         self.state0: NDArray[np.float64] | None = None # état initial
58.
59.         # Corps JWST (pour compatibilité avec le module visualization)
60.         self.jwst = Body(
61.             "JWST",
62.             mass=np.float64(6500),
63.             position=np.zeros(3, dtype=np.float64),
64.             velocity=np.zeros(3, dtype=np.float64),
65.         )
66.
67.     def run(self):
68.         """Lance l'intégration."""
69.         state0, T_half, c2 = richardson_halo_L2(
70.             self.Az, self.mu, self.northern, self.phi
71.         )
72.
73.         state0, self.T_halo = correct_halo(state0, self.mu, T_guess=2 * T_half)
74.         self.state0 = state0
75.
76.         t_end = self.n_revolutions * self.T_halo
77.         h = self.T_halo / self.n_steps_per_rev
78.
79.         f = eom_factory(self.mu)
80.         self.times, self.states = integrate(f, state0, ZERO, t_end, h)
81.         print(np.mean(np.linalg.norm(self.states[:, 3:], axis=1)))
```

Annexe – cr3bp/simulation.py

```
82. # Constante de Jacobi à chaque pas
83. self.jacobi = np.array(
84.     [jacobi_constant(self.states[i], self.mu) for i in range(len(self.times))],
85.     dtype=np.float64,
86. )
87.
88. # Mise à jour du corps JWST avec l'historique
89. self.jwst.clear_history()
90. for i in range(len(self.times)):
91.     self.jwst.position = self.states[i, :3]
92.     self.jwst.velocity = self.states[i, 3:]
93.     self.jwst.record()
94.
95. print(self._summary())
96.
97. def _summary(self) -> str:
98.     assert self.jacobi is not None and self.T_halo and self.times is not None
99.     dj = self.jacobi.max() - self.jacobi.min()
100.    J0 = self.jacobi[0]
101.    lines = [
102.        "\n[CR3BP] Simulation terminée",
103.        f" Révolutions : {self.n_revolutions}",
104.        f" Période halo : {self.T_halo:.4f} adim ",
105.        f" = {self.T_halo * T_STAR_SEC / 86400:.1f} jours",
106.        f" Pas h : {self.T_halo / self.n_steps_per_rev:.2e} adim",
107.        f" Pas totaux : {len(self.times)}",
108.        f" Jacobi CO : {J0:.6f}",
109.        f" ΔC (dérive) : {dj:.2e} ({100*dj/abs(J0):.4f} %)",
110.    ]
111.    return "\n".join(lines)
112.
113. @property
114. def positions(self) -> NDArray[np.float64]:
115.     """shape (N, 3) — positions adim. dans le repère tournant."""
116.     assert self.states is not None
117.     return self.states[:, :3]
118.
119. @property
120. def velocities(self) -> NDArray[np.float64]:
121.     """shape (N, 3) — vitesses adim."""
122.     assert self.states is not None
123.     return self.states[:, 3:]
124.
125. @property
126. def speeds(self) -> NDArray[np.float64]:
127.     """Norme de la vitesse, shape (N,)"""
128.     return np.linalg.norm(self.velocities, axis=1)
129.
130. @property
131. def times_days(self) -> NDArray[np.float64]:
132.     """Temps en jours."""
133.     assert self.times is not None
134.     return self.times * T_STAR_SEC / 86400
135.
136. def L2_position(self) -> NDArray[np.float64]:
137.     """Position de L2 dans le repère tournant [adim.]"""
138.     return lagrange_L2(self.mu)
```

Annexe – forces.py

```
1. import numpy as np
2. from core.body import Body
3. from numpy.typing import NDArray
4.
5. G = np.float64(6.674_30e-11) # constante gravitationnelle [m³ kg⁻¹ s⁻²]
6.
7. def gravitational_acceleration(
8.     pos_sc: NDArray[np.float64],
9.     bodies: list[Body],
10. ) -> NDArray[np.float64]:
11.     """
12.     Accélération gravitationnelle nette sur le spacecraft.
13.
14.      $a = \sum_i G m_i (r_i - r_{sc}) / |r_i - r_{sc}|^3$ 
15.
16.     Parameters
17.     -----
18.     pos_sc : NDArray, shape (3,) position du spacecraft [m]
19.     bodies : list of Body corps massifs (Soleil, Terre...)
20.
21.     Returns
22.     -----
23.     a : NDArray, shape (3,) [m/s²]
24.     """
25.     a = np.zeros(3, dtype=np.float64)
26.     for body in bodies:
27.         dr = body.position - pos_sc
28.         dist = np.linalg.norm(dr)
29.         if dist < 1.0:
30.             # Garde-fou numérique : évite la singularité (ne devrait jamais
31.             # arriver en pratique pour le JWST loin de tous les corps)
32.             continue
33.         a += G * body.mass * dr / dist**3
34.     return a
35.
36. def mechanical_energy(
37.     pos_sc: NDArray[np.float64],
38.     vel_sc: NDArray[np.float64],
39.     mass_sc: np.float64,
40.     bodies: list[Body],
41. ) -> np.float64:
42.     """
43.     Énergie mécanique spécifique du spacecraft [J/kg].
44.
45.      $E = v^2/2 - \sum_i G m_i / |r_i - r_{sc}|$ 
46.
47.     C'est l'énergie par unité de masse ; sa conservation est l'indicateur
48.     de qualité de l'intégration dans le référentiel inertiel.
49.     """
50.     kin = 0.5 * np.dot(vel_sc, vel_sc)
51.     pot = 0.0
52.     for body in bodies:
53.         dr = body.position - pos_sc
54.         dist = np.linalg.norm(dr)
55.         pot -= G * body.mass / dist
56.     return kin + pot
```

Annexe – inertial/simulation.py

```
1. import numpy as np
2. from core.body import Body
3. from core.integrator import integrate
4. from numpy.typing import NDArray
5.
6. from .forces import gravitational_acceleration, mechanical_energy
7.
8. # Constantes physiques SI
9. L_STAR = np.float64(1.495_978_707e11) # 1 UA [m]
10. M_SUN = np.float64(1.989e30) # [kg]
11. M_EARTH_MOON = np.float64(6.045e24)
12. T_EARTH_S = np.float64(365.25 * 86400) # Période orbitale Terre [s]
13. OMEGA_EARTH = np.float64(2 * np.pi / T_EARTH_S) # Vitesse angulaire [rad/s]
14. R_EARTH_ORBIT = np.float64(L_STAR) # Rayon orbite circulaire Terre = 1 UA
15.
16. class InertialSimulation:
17.     """
18.     Simulation N-corps du JWST dans le référentiel inertiel.
19.
20.     Parameters
21.     -----
22.     state0_cr3bp : NDArray, shape (6,)
23.         État initial du JWST en coordonnées CR3BP adim. (depuis Richardson).
24.         Sera converti automatiquement en SI.
25.     n_revolutions : np.float64
26.         Nombre de révolutions halo à simuler.
27.     T_halo_adim : np.float64
28.         Période halo en unités adim. (fournie par CR3BPSimulation).
29.     n_steps_per_rev : int
30.         Nombre de pas RK4 par révolution.
31.     """
32.
33.     def __init__(
34.         self,
35.         state0_cr3bp: NDArray[np.float64],
36.         n_revolutions: np.int16 = np.int16(4),
37.         T_halo_adim: np.float64 = np.float64(3), # valeur typique JWST
38.         n_steps_per_rev: np.int16 = np.int16(5000),
39.     ):
40.
41.         self.n_revolutions = n_revolutions
42.         self.T_halo_adim = T_halo_adim
43.         self.n_steps_per_rev = n_steps_per_rev
44.
45.         # Conversion t* → secondes
46.         self.T_star_s = T_EARTH_S / (2 * np.pi)
47.         self.T_halo_s = T_halo_adim * self.T_star_s
48.
49.         # Corps fixes (mis à jour dynamiquement à chaque pas)
50.         self.sun = Body(
51.             "Soleil",
52.             M_SUN,
53.             np.zeros(3, dtype=np.float64),
54.             np.zeros(3, dtype=np.float64),
55.             fixed=True,
56.         )
57.         self.earth = Body(
58.             "Terre",
59.             M_EARTH_MOON,
60.             np.zeros(3, dtype=np.float64),
61.             np.zeros(3, dtype=np.float64),
62.             fixed=True,
63.         )
64.
65.         # État initial JWST converti en SI
66.         self.state0_si = self._cr3bp_to_inertial(state0_cr3bp)
67.         self.jwst = Body(
68.             "JWST",
69.             mass=np.float64(6500),
70.             position=self.state0_si[:3],
71.             velocity=self.state0_si[3:],
72.         )
73.
74.         # Résultats
75.         self.times: NDArray[np.float64] | None = None
76.         self.states: NDArray[np.float64] | None = None
77.         self.energy: NDArray[np.float64] | None = None
78.
79.         # Conversion CR3BP adim. → inertiel SI (à t=0)
```

Annexe – inertial/simulation.py

```
79. def _cr3bp_to_inertial(
80.     self, state_cr3bp: NDArray[np.float64]
81. ) -> NDArray[np.float64]:
82.     """
83.     Convertit [x,y,z,vx,vy,vz] adim. CR3BP → SI inertiel à t=0.
84.
85.     À t=0 :
86.     - Le repère tournant est aligné avec le repère inertiel.
87.     - La vitesse dans le repère tournant inclut le terme de Coriolis
88.       ( $\omega \times r$ ) pour revenir au référentiel inertiel.
89.     """
90.     mu = M_EARTH_MOON / (M_SUN + M_EARTH_MOON)
91.
92.     x_nd, y_nd, z_nd, vx_nd, vy_nd, vz_nd = state_cr3bp
93.
94.     # Position SI (le barycentre est à l'origine dans le CR3BP)
95.     # Dans le CR3BP, le Soleil est à  $x = -\mu$ , la Terre à  $x = 1-\mu$ 
96.     # On recentre : l'origine inertielle est le Soleil (approximation,
97.     # car  $M_{\text{sun}} \gg M_{\text{earth}}$  le barycentre ≈ centre du Soleil)
98.     # Décalage du barycentre CR3BP par rapport au Soleil :  $x_{\text{bary}} = \mu * L_{\text{STAR}}$ 
99.     x_offset = -mu * L_STAR # ~450 km, négligeable mais inclus pour cohérence
100.
101.     pos_si = np.array(
102.         [
103.             x_nd * L_STAR - x_offset,
104.             y_nd * L_STAR,
105.             z_nd * L_STAR,
106.         ]
107.     )
108.
109.     # Vitesse : passage repère tournant → inertiel
110.     #  $v_{\text{inertiel}} = v_{\text{tournant}} + \omega \times r_{\text{tournant}}$ 
111.     #  $\omega = \omega_{\text{earth}} * \hat{z}$ ,  $r_{\text{tournant}} = (x_{\text{nd}}, y_{\text{nd}}, 0) * L_{\text{STAR}}$ 
112.     v_star = L_STAR / self.T_star_s # unité de vitesse adim. → SI
113.
114.     vx_si = vx_nd * v_star - OMEGA_EARTH * y_nd * L_STAR
115.     vy_si = vy_nd * v_star + OMEGA_EARTH * (x_nd * L_STAR - x_offset)
116.     vz_si = vz_nd * v_star
```

```
117.     return np.array(
118.         [pos_si[0], pos_si[1], pos_si[2], vx_si, vy_si, vz_si], dtype=np.float64
119.     )
120.
121. # Équations de mouvement N-corps (référentiel inertiel)
122. def _eom(self, t: np.float64, state: NDArray[np.float64]) -> NDArray[np.float64]:
123.     """
124.     dy/dt pour le vecteur d'état [x, y, z, vx, vy, vz] du JWST.
125.     Les corps massifs (Soleil, Terre) sont mis à jour à chaque appel.
126.     """
127.     # Mise à jour de la position de la Terre sur son orbite circulaire
128.     theta = OMEGA_EARTH * t
129.     self.earth.position = R_EARTH_ORBIT * np.array(
130.         [np.cos(theta), np.sin(theta), 0.0], dtype=np.float64
131.     )
132.
133.     # Soleil fixe à l'origine
134.     pos = state[:3]
135.     vel = state[3:]
136.     acc = gravitational_acceleration(pos, [self.sun, self.earth])
137.
138.     return np.concatenate([vel, acc], dtype=np.float64)
139.
140. # Lancement
141.
142. def run(self):
143.     t_end = self.n_revolutions * self.T_halo_s
144.     h = self.T_halo_s / self.n_steps_per_rev
145.
146.     self.times, self.states = integrate(
147.         self._eom, self.state0_si, np.float64(0), t_end, h
148.     )
149.
150.     # Énergie mécanique spécifique à chaque pas
151.     self.energy = np.empty(len(self.times), dtype=np.float64)
152.     for i, (t, s) in enumerate(zip(self.times, self.states)):
153.         theta = OMEGA_EARTH * t
154.         self.earth.position = R_EARTH_ORBIT * np.array(
155.             [np.cos(theta), np.sin(theta), 0.0], dtype=np.float64
156.         )
```

Annexe – inertial/simulation.py

```
157.     self.energy[i] = mechanical_energy(
158.         s[:3], s[3:], self.jwst.mass, [self.sun, self.earth]
159.     )
160.
161.     # Historique du corps JWST
162.     self.jwst.clear_history()
163.     for s in self.states:
164.         self.jwst.position = s[:3].copy()
165.         self.jwst.velocity = s[3:].copy()
166.         self.jwst.record()
167.
168.     print(self._summary())
169.
170. def _summary(self) -> str:
171.     assert self.energy is not None and self.T_halo_s and self.times is not None
172.     dE = self.energy.max() - self.energy.min()
173.     E0 = self.energy[0]
174.     lines = [
175.         "\n[Inertiel] Simulation terminée",
176.         f" Révolutions : {self.n_revolutions}",
177.         f" Période halo : {self.T_halo_s / 86400:.1f} jours",
178.         f" Pas h : {self.T_halo_s / self.n_steps_per_rev / 3600:.2f} h",
179.         f" Pas totaux : {len(self.times)}",
180.         f" E0 : {E0:.6e} J/kg",
181.         f" ΔE (dérive) : {dE:.2e} ({100*dE/abs(E0):.4f} %)",
182.     ]
183.     return "\n".join(lines)
184.
185. # Accesseurs
186. @property
187. def positions(self) -> NDArray[np.float64]:
188.     """shape (N, 3) [m]"""
189.     assert self.states is not None
190.     return self.states[:, :3]
191.
192. @property
193. def velocities(self) -> NDArray[np.float64]:
194.     """shape (N, 3) [m/s]"""
195.     assert self.states is not None
196.     return self.states[:, 3:]
197.
198. @property
199. def velocities(self) -> NDArray[np.float64]:
200.     """shape (N, 3) [m/s]"""
201.     assert self.states is not None
202.     return self.states[:, 3:]
203.
204. @property
205. def speeds(self) -> NDArray[np.float64]:
206.     """shape (N,) [m/s]"""
207.     return np.linalg.norm(self.velocities, axis=1)
208.
209. @property
210. def times_days(self) -> NDArray[np.float64]:
211.     assert self.times is not None
212.     return self.times / 86400
213.
214. def earth_positions(self) -> NDArray[np.float64]:
215.     """Positions de la Terre à chaque instant, shape (N, 3) [m]."""
216.     assert self.times is not None
217.     theta = OMEGA_EARTH * self.times
218.     return R_EARTH_ORBIT * np.column_stack(
219.         [
220.             np.cos(theta, dtype=np.float64),
221.             np.sin(theta, dtype=np.float64),
222.             np.zeros_like(theta, dtype=np.float64),
223.         ]
224.     )
```

Annexe – cr3bp/stationkeeping.py

```
1. from dataclasses import dataclass
2. from typing import Callable
3.
4. import numpy as np
5. from core.integrator import integrate, rk4_step
6. from numpy.typing import NDArray
7.
8. from .equations import MU_SUN_EARTH, eom_factory, jacobi_constant
9. from .lagrange import richardson_halo_L2
10. from .stm import (correct_halo, eom_stm_factory, jacobian,
11.                  print_monodromy_summary, stable_unstable_eigvecs)
12.
13. T_STAR_SEC = np.float64(365.25 * 86400 / (2 * np.pi))
14. V_STAR_MS = np.float64(1.496e11 / T_STAR_SEC)
15. ZERO = np.float64(0)
16.
17. # Structure d'une manœuvre
18. @dataclass
19. class Maneuver:
20.     t_adim: np.float64
21.     t_days: np.float64
22.     delta_v: NDArray[np.float64]
23.     dv_norm: np.float64
24.     dv_norm_ms: np.float64
25.     state_before: NDArray[np.float64]
26.     state_after: NDArray[np.float64]
27.     error_before: NDArray[np.float64]
28.
29. # ΔV EVSK avec vecteurs propres locaux (propagés par STM)
30. def _local_eigvecs(
31.     Phi_k: NDArray[np.float64],
32.     v_s0: NDArray[np.float64],
33.     v_u_left0: NDArray[np.float64],
34.     v_u0: NDArray[np.float64],
35. ) -> tuple[NDArray[np.float64], NDArray[np.float64]]:
36.     """
37.     Propage v_s et v_u_left au temps t_k via la STM.
38.
39.     v_s(t_k) = Φ(t_k) · v_s(0) (stable droit)
40.     v_u_left(t_k) = Φ^{-T}(t_k) · v_u_left(0) (instable gauche)
```

```
41.     La normalisation biorthogonale ⟨v_u_left, v_u⟩ = 1 est préservée
42.     par
43.     cette propagation (propriété de la STM), on renormalise quand
44.     même pour
45.     éviter l'accumulation d'erreurs numériques.
46.
47.     Parameters
48.     -----
49.     Phi_k : (6,6) STM Φ(t_k, 0)
50.     v_s0 : (6,) vecteur propre stable à t=0
51.     v_u_left0 : (6,) vecteur propre instable gauche à t=0
52.     (biorthonorm.)
53.     v_u0 : (6,) vecteur propre instable droit à t=0
54.     """
55.
56.     # Propagation du vecteur stable
57.     v_s_k = Phi_k @ v_s0
58.     norm_s = np.float64(np.linalg.norm(v_s_k))
59.     if norm_s > 1e-14:
60.         v_s_k /= norm_s
61.
62.     # Propagation du vecteur instable gauche : Φ^{-T} w = solve(Φ^T,
63.     w)
64.     v_u_left_k = np.linalg.solve(Phi_k.T, v_u_left0)
65.
66.     # Propagation du vecteur instable droit (pour
67.     biorthonormalisation)
68.     v_u_k = Phi_k @ v_u0
69.     dot = np.float64(np.dot(v_u_left_k, v_u_k))
70.     if abs(dot) > 1e-14:
71.         v_u_left_k /= dot
72.
73.     return v_s_k, v_u_left_k
```

Annexe – cr3bp/stationkeeping.py

```

68. def evsk_delta_v(
69.     dx: NDArray[np.float64],
70.     v_s: NDArray[np.float64],
71.     v_u_left: NDArray[np.float64],
72.     dv_max_adim: np.float64 = np.float64(1e-4),
73. ) -> NDArray[np.float64]:
74.     """
75.     Calcule le vecteur  $\Delta v$  selon l'algorithme EVSK.
76.
77.     Parameters
78.     -----
79.     dx          : (6,)   écart d'état  $\delta x = x_{\text{réel}} - x_{\text{réf}}$ 
80.     v_s         : (6,)   vecteur propre stable LOCAL (propagé par STM)
81.     v_u_left    : (6,)   vecteur propre instable gauche LOCAL
82.     dv_max_adim : np.float64 plafond de sécurité sur  $|\Delta v|$  [adim.
83.     vitesse]
84.
85.     Returns
86.     -----
87.     dv : (3,)   impulsion en vitesse [adim.]
88.     """
89.     e_v = v_u_left[3:].copy()
90.     norm_ev = np.float64(np.linalg.norm(e_v))
91.     if norm_ev < 1e-14:
92.         return np.zeros(3, dtype=np.float64)
93.     e_v /= norm_ev
94.
95.     dv_state = np.zeros(6, dtype=np.float64)
96.     dv_state[3:] = e_v
97.
98.     proj_dx = np.dot(v_u_left, dx)
99.     proj_ev = np.dot(v_u_left, dv_state)
100.
101.     if abs(proj_ev) < 1e-14:
102.         return np.zeros(3)
103.
104.     alpha = -proj_dx / proj_ev
105.
106.     if abs(alpha) > dv_max_adim:
107.         alpha = np.sign(alpha) * dv_max_adim

```

```

107.     return alpha * e_v
108.
109. # Simulation avec station-keeping
110. class StationKeepingSimulation:
111.     """
112.     Simulation CR3BP avec corrections EVSK (vecteurs propres propagés
    par STM).
113.
114.     Parameters
115.     -----
116.     Az : np.float64
117.         Amplitude hors-plan [adim.].
118.     mu : np.float64
119.         Paramètre de masse CR3BP.
120.     n_revolutions : np.float64
121.         Durée de simulation [révolutions halo].
122.     n_steps_per_rev : int
123.         Pas RK4 par révolution.
124.     dt_maneuver_days : np.float64
125.         Intervalle entre manœuvres [jours]. Défaut : 21 j.
126.     perturbation : array (6,) or None
127.         Perturbation initiale. Défaut : +100 km radial.
128.     dv_max_ms : np.float64
129.         Plafond de sécurité sur  $|\Delta V|$  [m/s].
130.     n_stm_steps : int
131.         Pas pour la monodromie et la STM de référence.
132.     """
133.
134.     def __init__(
135.         self,
136.         Az: np.float64 = np.float64(0.00279),
137.         mu: np.float64 = MU_SUN_EARTH,
138.         n_revolutions: np.int16 = np.int16(4),
139.         n_steps_per_rev: np.int16 = np.int16(5000),
140.         dt_maneuver_days: np.float32 = np.float32(21),
141.         perturbation: NDArray[np.float64] | None = None,
142.         dv_max_ms: np.float64 = np.float64(2.0),
143.         n_stm_steps: np.int16 = np.int16(10000),
144.     ):

```

Annexe – cr3bp/stationkeeping.py

```
145. self.Az = Az
146. self.mu = mu
147. self.n_revolutions = n_revolutions
148. self.n_steps_per_rev = n_steps_per_rev
149. self.n_stm_steps = n_stm_steps
150. self.northern = False
151. self.dt_maneuver_days = dt_maneuver_days
152. self.dt_maneuver = np.float64(dt_maneuver_days * 86400 /
    T_STAR_SEC)
153. self.dv_max_adim = dv_max_ms / V_STAR_MS
154. self._perturbation = perturbation
155.
156. self.times: NDArray[np.float64] | None = None
157. self.states: NDArray[np.float64] | None = None
158. self.states_free: NDArray[np.float64] | None = None
159. self.states_ref: NDArray[np.float64] | None = None
160. self.jacobi: NDArray[np.float64] | None = None
161. self.maneuvers: list[Maneuver] = []
162.
163. self.M: NDArray[np.float64] | None = None
164. self.v_s: NDArray[np.float64] | None = None
165. self.v_s_list: list[NDArray[np.float64]] = []
166. self.v_u: NDArray[np.float64] | None = None
167. self.v_u_left: NDArray[np.float64] | None = None
168. self.lam_s: complex | None = None
169. self.lam_u: complex | None = None
170. self.T_halo: np.float64 | None = None
171. self.state0_ref: NDArray[np.float64] | None = None
172.
173. # STM dense sur une période (pour propagation des vecteurs
    propres)
174. self._t_stm: NDArray[np.float64] | None = None # shape
    (N_stm,)
175. self._stm_arr: NDArray[np.float64] | None = None # shape
    (N_stm, 6, 6)
176.
177. def run(self):
178.     # 1. État initial et période
179.     state0, T_half, c2 = richardson_halo_L2(
180.         self.Az, self.mu, northern=self.northern, phi=ZERO
181.
182.     # Raffinement en orbite périodique (tir simple) AVANT la
    monodromie : sinon
183.     # M =  $\Phi(T,0)$  n'a pas la structure  $\{1, 1, \lambda, 1/\lambda, e^{\pm i0}\}$ 
    (valeurs propres
184.     # toutes complexes) car la graine n'est pas réellement
    périodique.
185.     state0, self.T_halo = correct_halo(state0, self.mu, T_guess=2
    * T_half)
186.     self.state0_ref = state0
187.     t_end = self.n_revolutions * self.T_halo
188.     h = self.T_halo / self.n_steps_per_rev
189.     f = eom_factory(self.mu)
190.
191.     # 2. Propagation référence + STM sur une période (système
    augmenté 42D)
192.     print("[SK] Propagation référence + STM sur une période...")
193.     y0_stm = np.concatenate([state0,
194.         np.eye(6).flatten(order="F")])
195.     t_stm, ys_stm = integrate(
196.         eom_stm_factory(self.mu),
197.         y0_stm,
198.         ZERO,
199.         self.T_halo,
200.         self.T_halo / self.n_stm_steps,
201.     )
202.     self._t_stm = t_stm
203.     self._stm_arr = ys_stm[:, 6:].reshape(-1, 6, 6, order="F")
204.     # 3. Monodromie et vecteurs propres à t=0
205.     self.M = self._stm_arr[-1]
206.     assert self.M is not None
207.     self.v_s, self.v_u, _, self.v_u_left, self.lam_s, self.lam_u
    = (
208.         stable_unstable_eigvecs(self.M)
209.     )
210.     print_monodromy_summary(self.M, self.mu)
```

Annexe – cr3bp/stationkeeping.py

```
208.     # 4. Propagation référence sur toute la durée (pour
      interpolation 0x)
209.     print("[SK] Propagation de l'orbite de référence
      complète...")
210.     t_ref, s_ref = integrate(f, state0, ZERO, t_end, h)
211.     self._t_ref = t_ref
212.     self._s_ref = s_ref
213.
214.     # 5. État initial perturbé
215.     if self._perturbation is not None:
216.         s0_p = state0 + self._perturbation
217.     else:
218.         pert = np.zeros(6)
219.         pert[0] = 100e3 / 1.496e11 # +100 km radial
220.         s0_p = state0 + pert
221.
222.     # 6. Intégration avec corrections EVSK
223.     print("[SK] Intégration avec corrections EVSK...")
224.     self.times, self.states = self._integrate_with_sk(s0_p,
      t_end, h, f)
225.
226.     # 7. Intégration libre (comparaison)
227.     _, self.states_free = integrate(f, s0_p, ZERO, t_end, h)
228.
229.     # 8. Référence interpolée
230.     self.states_ref = np.array([self._ref_state(t) for t in
      self.times], np.float64)
231.
232.     # 9. Constante de Jacobi
233.     self.jacobi = np.array(
234.         [jacobi_constant(self.states[i], self.mu) for i in
      range(len(self.times))]
235.     )
236.
237.     print(self._summary())
238.
239.     def _ref_state(self, t: np.float64) -> NDArray[np.float64]:
240.         """Interpolation de la référence à l'instant t."""
241.         assert self.T_halo is not None
242.         t_mod = t
```

```
243.         idx = np.clip(
244.             np.searchsorted(self._t_ref, t_mod),
245.             1,
246.             len(self._t_ref) - 1,
247.             dtype=np.int64,
248.         )
249.         t0, t1 = self._t_ref[idx - 1], self._t_ref[idx]
250.         s0, s1 = self._s_ref[idx - 1], self._s_ref[idx]
251.         if t1 == t0:
252.             return s0
253.         return s0 + (t - t0) / (t1 - t0) * (s1 - s0)
254.
255.     def _stm_at(self, t: np.float64) -> NDArray[np.float64]:
256.         """
257.         STM  $\Phi(t \bmod T, 0)$  par interpolation sur le tableau pré-
      calculé.
258.         On utilise  $t \bmod T_{\text{halo}}$  car la STM est périodique
      (approximativement).
259.         """
260.
261.         assert (
262.             self._t_stm is not None
263.             and self._stm_arr is not None
264.             and self.T_halo is not None
265.         )
266.
267.         t_mod = t % self.T_halo
268.         n = int(t // self.T_halo)
269.
270.         idx = np.clip(
271.             np.searchsorted(self._t_stm, t_mod),
272.             1,
273.             len(self._t_stm) - 1,
274.             dtype=np.int64,
275.         )
276.         t0, t1 = self._t_stm[idx - 1], self._t_stm[idx]
277.         P0, P1 = self._stm_arr[idx - 1], self._stm_arr[idx]
278.         if t1 == t0:
279.             return P0
280.         alpha = (t_mod - t0) / (t1 - t0)
281.
282.         Mn = np.linalg.matrix_power(self.M, n)
283.         return (P0 + alpha * (P1 - P0)) @ Mn
```

Annexe – cr3bp/stationkeeping.py

```
283.     def _integrate_with_sk(
284.         self, s0: NDArray[np.float64], t_end, h: np.float64, f:
285.         Callable
286.     ):
287.         n_steps = np.int64(np.ceil(t_end / h))
288.         times = np.empty(n_steps + 1)
289.         states = np.empty((n_steps + 1, 6), dtype=np.float64)
290.
291.         t, s = ZERO, s0.copy()
292.         times[0], states[0] = t, s
293.         t_next_man = self.dt_maneuver
294.
295.         for i in range(1, n_steps + 1):
296.             h_eff = min(h, t_end - t)
297.
298.             if t < t_next_man <= t + h_eff:
299.                 h1 = t_next_man - t
300.                 s = rk4_step(f, t, s, h1)
301.                 t = t_next_man
302.                 s = self._apply_maneuver(t, s)
303.                 h2 = h_eff - h1
304.                 if h2 > 1e-15:
305.                     s = rk4_step(f, t, s, h2)
306.                     t += h2
307.                 t_next_man += self.dt_maneuver
308.             else:
309.                 s = rk4_step(f, t, s, h_eff)
310.                 t += h_eff
311.
312.             times[i], states[i] = t, s
313.
314.         return times, states
315.
316.     def _apply_maneuver(
317.         self, t: np.float64, state: NDArray[np.float64]
318.     ) -> NDArray[np.float64]:
319.         x_ref = self._ref_state(t)
320.         dx = state - x_ref
321.
322.         # Vecteurs propres propagés au temps t
323.         Phi_k = self._stm_at(t)
324.
325.         assert (
326.             self.v_s is not None and self.v_u_left is not None and
327.             self.v_u is not None
328.         )
329.         v_s_k, v_u_left_k = _local_eigvecs(Phi_k, self.v_s,
330.             self.v_u_left, self.v_u)
331.         self.v_s_list.append(v_s_k)
332.
333.         dv = evsk_delta_v(dx, v_s_k, v_u_left_k, self.dv_max_adim)
334.
335.         state_after = state.copy()
336.         state_after[3:] += dv
337.
338.         self.maneuvers.append(
339.             Maneuver(
340.                 t_adim=t,
341.                 t_days=t * T_STAR_SEC / 86400,
342.                 delta_v=dv,
343.                 dv_norm=np.float64(np.linalg.norm(dv)),
344.                 dv_norm_ms=np.float64(np.linalg.norm(dv) *
345.                     V_STAR_MS),
346.                 state_before=state.copy(),
347.                 state_after=state_after.copy(),
348.                 error_before=dx.copy(),
349.             )
350.         )
351.         return state_after
352.
353.     @property
354.     def positions(self) -> NDArray[np.float64]:
355.         assert self.states is not None
356.         return self.states[:, :3]
357.
358.     @property
359.     def positions_free(self) -> NDArray[np.float64]:
360.         assert self.states_free is not None
361.         return self.states_free[:, :3]
362.
363.     @property
364.     def positions_ref(self) -> NDArray[np.float64]:
365.         assert self.states_ref is not None
366.         return self.states_ref[:, :3]
```

Annexe – cr3bp/stationkeeping.py

```
262. @property
263. def times_days(self) -> NDArray[np.float64]:
264.     assert self.times is not None
265.     return self.times * (T_STAR_SEC / 86400)
266.
267. @property
268. def dv_norms_ms(self) -> NDArray[np.float64]:
269.     return np.array([m.dv_norm_ms for m in self.maneuvers],
np.float64)
270.
271. @property
272. def maneuver_times_days(self) -> NDArray[np.float64]:
273.     return np.array([m.t_days for m in self.maneuvers],
np.float64)
274.
275. @property
276. def total_dv_ms(self) -> np.float64:
277.     return np.float64(np.sum(self.dv_norms_ms))
278.
279. @property
280. def position_errors(self) -> NDArray[np.float64]:
281.     return np.linalg.norm(self.positions - self.positions_ref,
axis=1) * 1.496e8
282.
283. @property
284. def position_errors_free(self) -> NDArray[np.float64]:
285.     return (
286.         np.linalg.norm(self.positions_free - self.positions_ref,
axis=1) * 1.496e8
287.     )
288.
289. def _summary(self) -> str:
290.     n = len(self.maneuvers)
291.     dvs = self.dv_norms_ms if n > 0 else np.array([0],
np.float64)
292.     lines = [
293.         "\n[Station-Keeping] Simulation terminée",
294.         f" Révolutions : {self.n_revolutions}",
295.         f" Intervalle manoeuv.: {self.dt_maneuver_days} jours",
296.         f" Nombre de manoeuvres: {n}",
297.     ]
298.     if n > 0:
299.         lines += [
300.             f" ΔV min/max/moy : {dvs.min():.4f} /
{dvs.max():.4f} / {dvs.mean():.4f} m/s",
301.             f" ΔV total cumulé : {self.total_dv_ms:.4f} m/s",
302.             f" Erreur pos. max SK :
{self.position_errors.max():.1f} km",
303.             f" Erreur pos. max lib:
{self.position_errors_free.max():.1f} km",
304.         ]
305.     return "\n".join(lines)
```

Annexe – Barycentre.py

```
1. import numpy as np
2. import matplotlib.pyplot as plt
3.
4. MU = 3.040423389123456e-6 # valeur JPL
5.
6. # Positions adim. dans le repère tournant
7. x_sun = -MU
8. x_earth = 1.0 - MU
9. x_bary = 0.0
10.
11. # Distances en km pour les annotations
12. UA_KM = 1.496e8
13. d_sun_bary = abs(x_sun) * UA_KM
14. d_earth_bary = x_earth * UA_KM
15.
16. print(f"μ = {MU:.4e}")
17. print(f"x_Soleil = {x_sun:.4e} adim. ({d_sun_bary:.0f} km du barycentre)")
18. print(f"x_Terre = {x_earth:.8f} adim. ({d_earth_bary:.0f} km du barycentre)")
19. print(f"x_barycentre = {x_bary} (origine du repère)")
20. print(f"Distance Soleil-Terre= {UA_KM:.4e} km = 1 UA")
21. print(f"Vérification : μ·x_T + (1-μ)·x_S = {MU*x_earth + (1-MU)*x_sun:.2e} (doit être 0)")
22.
23. # — Figure —
24. fig, ax = plt.subplots(figsize=(12, 4), facecolor="#0D0D1A")
25. ax.set_facecolor("#0D0D1A")
26.
27. # Axe horizontal (ligne de base)
28. ax.axhline(0, color="#2A2A3E", lw=0.8, zorder=0)
29.
30. # Soleil
31. ax.scatter(x_sun, 0, s=1200, color="#FFD700", zorder=5, edgecolors="#FFF4A0",
32.            linewidths=1.5)
33. ax.annotate(
34.     f"Soleil\nx = -μ = {x_sun:.2e}",
35.     xy=(x_sun, 0), xytext=(x_sun - 0.1, 0.018),
36.     color="#FFD700", fontsize=9, fontfamily="monospace",
37.     arrowprops=dict(arrowstyle="→", color="#FFD700", lw=0.8)
38. )
39. # Terre
40. ax.scatter(x_earth, 0, s=180, color="#3A9BD5", zorder=5, edgecolors="#A0D8F0",
41.            linewidths=1.2)
42. ax.annotate(
43.     f"Terre\nx = 1-μ = {x_earth:.6f}",
44.     xy=(x_earth, 0), xytext=(x_earth - 0.15, -0.018),
45.     color="#3A9BD5", fontsize=9, fontfamily="monospace",
46.     arrowprops=dict(arrowstyle="→", color="#3A9BD5", lw=0.8)
47. )
48. # Barycentre
49. ax.scatter(x_bary, 0, s=60, color="#FF6B6B", marker="+", linewidths=2.0, zorder=6)
50. ax.annotate(
51.     f"Barycentre\nx = 0 (origine)",
52.     xy=(x_bary, 0), xytext=(0.08, 0.018),
53.     color="#FF6B6B", fontsize=9, fontfamily="monospace",
54.     arrowprops=dict(arrowstyle="→", color="#FF6B6B", lw=0.8)
55. )
56.
57. # Cote μ : distance Soleil → barycentre
58. ax.annotate("", xy=(0, -0.03), xytext=(x_sun, -0.03),
59.            color="#FFD700", lw=1.0))
60. ax.text((x_sun + 0)/2, -0.038, f"μ = {MU:.2e}", color="#FFD700", ha="center",
61.        fontfamily="monospace")
62.
63. # Cote 1-μ : distance barycentre → Terre
64. ax.annotate("", xy=(x_earth, -0.03), xytext=(0, -0.03),
65.            color="#3A9BD5", lw=1.0))
66. ax.text(x_earth/2, -0.038, f"1-μ = {1-MU:.6f}", color="#3A9BD5",
67.        fontfamily="monospace")
68.
69. # Cote totale : 1 UA
70. ax.annotate("", xy=(x_earth, 0.035), xytext=(x_sun, 0.035),
71.            arrowprops=dict(arrowstyle="↔", color="#E0E0E0", lw=0.8))
72. ax.text((x_sun + x_earth)/2, 0.042, "1 UA (unité adim.)",
73.        color="#E0E0E0", ha="center",
74.        fontfamily="monospace")
```

Annexe – Barycentre.py

```
74. ax.set_xlim(-0.15, 1.15)
75. ax.set_ylim(-0.07, 0.07)
76. ax.set_xlabel("x [adim., 1 = 1 UA]", color="#E0E0E0", fontfamily="monospace")
77. ax.set_yticks([])
78.
79. ax.tick_params(colors="#E0E0E0")
80. ax.set_title(
81.     f"Repère tournant CR3BP — Soleil, Terre, Barycentre\n"
82.     f"Le barycentre est à {d_sun_bary:.0f} 000 km du centre du Soleil "
83.     f"(≈ {d_sun_bary / 6.957e5:.5f} R☉)",
84.     color="#E0E0E0", fontfamily="monospace", fontsize=10
85. )
86. for spine in ax.spines.values():
87.     spine.set_edgecolor("#2A2A3E")
88.
89. plt.tight_layout()
90. plt.show()
```

Annexe – cr3bp_potential.py

```
1. import matplotlib as mpl
2. import matplotlib.pyplot as plt
3. import numpy as np
4. from numpy.typing import NDArray
5. from scipy.optimize import brentq
6.
7. COLORS = {
8.     "earth": "#3A9BD5",
9.     "L2": "#00D9FF",
10.    "bg": "#FFFFFF",
11.    "text": "#0D0D1A",
12.    "grid": "#2A2A3E",
13. }
14.
15. mu = 3.003e-6 # Soleil-Terre
16.
17. def potential(X: NDArray[np.float64], Y: NDArray[np.float64]) -> NDArray[np.float64]:
18.     D = np.sqrt((X + mu) ** 2 + Y**2)
19.     R = np.sqrt((X - 1 + mu) ** 2 + Y**2)
20.     D = np.where(D < 1e-5, 1e-5, D)
21.     R = np.where(R < 1e-5, 1e-5, R)
22.     return -((1 - mu) / D + mu / R + 0.5 * (X**2 + Y**2))
23.
24. def find_L2():
25.     def dUdx(x):
26.         d = abs(x + mu)
27.         r = abs(x - 1 + mu)
28.         return x - (1 - mu) * np.sign(x + mu) / d**2 - mu * np.sign(x - 1 + mu) / r**2
29.
30.     return brentq(dUdx, 1 - mu + 1e-7, 1.05)
31.
32. x_L2 = find_L2()
33.
34. # Grille zoomée autour de L2
35. N = 900
36. xlim = (0.98, 1.02)
37. ylim = (-0.05, 0.05)
38. x = np.linspace(*xlim, N)
39. y = np.linspace(*ylim, N)
40. X, Y = np.meshgrid(x, y)
41. U = potential(X, Y)
42. U_L2 = potential(np.array([x_L2], dtype=np.float64), np.array([0.0], dtype=np.float64))
43. 0
44. ]
45.
46. # Equipotentielles : niveaux entre min et 99e percentile, strictement croissants
47. u_min = np.nanpercentile(U, 2)
48. u_max = np.nanpercentile(U, 99)
49. levels = np.unique(np.linspace(u_min, u_max, 30))
50.
51. # Figure
52. mpl.rcParams.update(
53.     {
54.         "figure.facecolor": COLORS["bg"],
55.         "axes.facecolor": COLORS["bg"],
56.         "axes.edgecolor": COLORS["text"],
57.         "axes.labelcolor": COLORS["text"],
58.         "xtick.color": COLORS["text"],
59.         "ytick.color": COLORS["text"],
60.         "text.color": COLORS["text"],
61.         "grid.color": COLORS["grid"],
62.         "grid.linestyle": "--",
63.         "grid.alpha": 0.4,
64.         "font.family": "monospace",
65.         "font.size": 15,
66.         "axes.titlesize": 20,
67.         "axes.titleweight": "bold",
68.     }
69. )
70.
71. fig, ax = plt.subplots(figsize=(10, 10))
72.
73. # Heatmap
74. im = ax.pcolormesh(X, Y, np.clip(U, u_min, u_max), shading="gouraud", cmap="plasma")
75. cbar = fig.colorbar(im, ax=ax, pad=0.02, fraction=0.025)
76. cbar.set_label(r"$U^{(x,y)}$ [u. norm.]", color=COLORS["text"])
77. plt.setp(cbar.ax.yaxis.get_ticklabels(), color=COLORS["text"])
78.
79. # Equipotentielles régulières
80. ax.contour(X, Y, U, levels=levels, colors="white", linewidths=0.4, alpha=0.4)
```

Annexe – cr3bp_potential.py

```
81. # Equipotentielles régulières
82. ax.contour(X, Y, U, levels=levels, colors="white", linewidths=0.4, alpha=0.4)
83.
84. # Equipotentielle critique en L2
85. # ax.contour(X, Y, U, levels=[U_L2 * 0.999995], colors=COLORS["L2"], linewidths=1.8,
    linestyle="--")
86.
87. # Terre (hors champ à gauche, on indique juste avec une flèche)
88. ax.annotate("Terre", xy=(1 - mu, 0.005), color="#000000", fontsize=15, va="center")
89.
90. ax.plot(1 - mu, 0, "o", color=COLORS["earth"], ms=14, zorder=6)
91.
92. # L2
93. ax.plot(x_L2, 0, "*", color=COLORS["L2"], ms=14, zorder=6)
94. ax.annotate(
95.     f"$L_2$\n$x={x_L2:.7f}$",
96.     xy=(x_L2, 0),
97.     xytext=(x_L2 + 0.001, 0.009),
98.     color=COLORS["L2"],
99.     fontsize=15,
100.    fontweight="bold",
101.    arrowprops=dict(arrowstyle="-", color=COLORS["L2"], lw=0.8),
102. )
103.
104. ax.set_xlim(xlim)
105. ax.set_ylim(ylim)
106. ax.set_xlabel("$x$ [u. norm.]")
107. ax.set_ylabel("$y$ [u. norm.]")
108. ax.set_title(r"Potentiel effectif $U^*(x,y)$ — CR3BP Soleil-Terre, voisinage de $L_2$")
109. ax.grid(True, alpha=0.25)
110.
111. plt.tight_layout()
112. plt.show()
113. # fig.savefig("/mnt/user-data/outputs/cr3bp_potential.png", dpi=180, bbox_inches="tight")
114. print("OK")
```

Annexe – utils.py

```
1. import matplotlib as mpl
2. import matplotlib.pyplot as plt
3. import numpy as np
4. from numpy.typing import NDArray
5.
6. # Palette
7. COLORS = {
8.     "jwst": "#00BFFF", # bleu ciel
9.     "earth": "#3A9BD5", # bleu Terre
10.    "sun": "#FFD700", # or Soleil
11.    "L2": "#FF6B6B", # rouge L2
12.    "energy": "#90EE90", # vert énergie
13.    "jacobi": "#DDA0DD", # violet Jacobi
14.    "speed": "#FFA07A", # orange vitesse
15.    "grid": "#2A2A3E", # "#636374",
16.    "bg": "#FFFFFF", # "#E0E0E0",
17.    "text": "#0D0D1A",
18. }
19.
20. # Thème global
21. def set_style():
22.     """Applique le thème à tous les graphes."""
23.     mpl.rcParams.update(
24.         {
25.             "figure.facecolor": COLORS["bg"],
26.             "axes.facecolor": COLORS["bg"],
27.             "axes.edgecolor": COLORS["text"],
28.             "axes.labelcolor": COLORS["text"],
29.             "xtick.color": COLORS["text"],
30.             "ytick.color": COLORS["text"],
31.             "text.color": COLORS["text"],
32.             "grid.color": COLORS["grid"],
33.             "grid.linestyle": "--",
34.             "grid.alpha": 0.4,
35.             "legend.facecolor": "#FFFFFF", # "#1A1A2E",
36.             "legend.edgecolor": COLORS["text"],
37.             "font.family": "monospace",
38.             "font.size": 10,
39.             "axes.titlesize": 12,
40.             "axes.titleweight": "bold",
41.             "lines.linewidth": 1.2,
42.         }
43.     )
44.
45. # Helpers
46. def make_fig(nrows: int = 1, ncols: int = 1, **kwargs) -> tuple:
47.     """Crée une figure avec le thème appliqué."""
48.     set_style()
49.     fig, axes = plt.subplots(nrows, ncols, **kwargs)
50.     return fig, axes
51.
52. def add_colorbar_time(fig, ax, scatter, label: str = "Temps"):
53.     """Ajoute une colorbar temporelle à un scatter plot."""
54.     cb = fig.colorbar(scatter, ax=ax, pad=0.02, fraction=0.03)
55.     cb.set_label(label, color=COLORS["text"])
56.     cb.ax.yaxis.set_tick_params(color=COLORS["text"])
57.     plt.setp(cb.ax.yaxis.get_ticklabels(), color=COLORS["text"])
58.
59. def relative_drift(arr: NDArray[np.float64]) -> np.float64:
60.     """Drift relatif d'une quantité censée être conservée."""
61.     return (arr.max() - arr.min()) / abs(arr[0])
62.
63. def annotate_extrema(
64.     ax, x: NDArray[np.float64], y: NDArray[np.float64], label: str = "", n: int = 1
65. ):
66.     """Annote les n extrema globaux (max et min) sur un axe."""
67.     idx_max = np.argmax(y)
68.     idx_min = np.argmin(y)
69.     for idx, tag in [(idx_max, "max"), (idx_min, "min")]:
70.         ax.annotate(
71.             f"{label}{tag}={y[idx]:.3e}",
72.             xy=(x[idx], y[idx]),
73.             xytext=(10, 10),
74.             textcoords="offset points",
75.             color=COLORS["text"],
76.             fontsize=8,
77.             arrowprops=dict(arrowstyle="->", color=COLORS["text"], lw=0.8),
78.         )
```

Annexe – trajectory.py

```
1. import matplotlib.pyplot as plt
2. from cr3bp.simulation import CR3BPSimulation
3. from inertial.simulation import InertialSimulation
4. from mpl_toolkits.mplot3d import \
5.     Axes3D # noqa: F401 (enregistrement projection 3d)
6.
7. from .utils import COLORS, add_colorbar_time, set_style
8.
9. # CR3BP
10. def plot_cr3bp_trajectory(sim: CR3BPSimulation, save_path: str | None = None):
11.     """
12.     Trajectoire 3D + projections 2D dans le repère tournant centré sur L2.
13.
14.     Parameters
15.     -----
16.     sim : CR3BPSimulation (après run())
17.     """
18.     assert sim.times is not None
19.
20.     set_style()
21.     pos = sim.positions
22.     L2 = sim.L2_position()
23.
24.     # Coordonnées relatives à L2 [km]
25.     KM = 1.495_978_707e8 # 1 unité adim. → km (1 UA en km)
26.     dpos = (pos - L2) * KM
27.
28.     t_norm = sim.times / sim.times[-1] # [0, 1] pour la colormap
29.
30.     fig = plt.figure(figsize=(14, 10))
31.     fig.suptitle("JWST — Orbite halo autour de L2 (CR3BP, repère tournant)", y=0.98)
32.
33.     # --- 3D ---
34.     ax3 = fig.add_subplot(2, 2, 1, projection="3d")
35.     ax3.set_facecolor(COLORS["bg"])
36.     sc = ax3.scatter(
37.         dpos[:, 0], dpos[:, 1], dpos[:, 2], c=t_norm, cmap="plasma", s=0.5, alpha=0.8
38.     )
39.     ax3.scatter(0, 0, 0, color=COLORS["L2"], s=60, zorder=5, label="L2")
40.     ax3.scatter(-1.5e6, 0, 0, color=COLORS["earth"], s=60, zorder=5, label="Terre")
41.
42.     ax3.set_xlabel("ΔX [km]")
43.     ax3.set_ylabel("ΔY [km]")
44.     ax3.set_zlabel("ΔZ [km]")
45.     ax3.set_title("Vue 3D")
46.     ax3.legend(fontsize=8)
47.
48.     # --- XY ---
49.     ax_xy = fig.add_subplot(2, 2, 2)
50.     sc2 = ax_xy.scatter(dpos[:, 0], dpos[:, 1], c=t_norm, cmap="plasma", s=0.5)
51.     ax_xy.scatter(0, 0, color=COLORS["L2"], s=60, zorder=5, label="L2")
52.     ax_xy.scatter(-1.5e6, 0, color=COLORS["earth"], s=60, zorder=5, label="Terre")
53.     ax_xy.set_xlabel("ΔX [km]")
54.     ax_xy.set_ylabel("ΔY [km]")
55.     ax_xy.set_title("Plan XY (écliptique)")
56.     ax_xy.set_aspect("equal")
57.     ax_xy.grid(True)
58.     ax_xy.legend(fontsize=8)
59.     add_colorbar_time(fig, ax_xy, sc2, label=f"Temps [0 → {sim.times_days[-1]:.0f} j]")
60.
61.     # --- XZ ---
62.     ax_xz = fig.add_subplot(2, 2, 3)
63.     ax_xz.axis("equal")
64.     ax_xz.scatter(dpos[:, 0], dpos[:, 2], c=t_norm, cmap="plasma", s=0.5)
65.     ax_xz.scatter(0, 0, color=COLORS["L2"], s=60, zorder=5)
66.     ax_xz.scatter(-1.5e6, 0, color=COLORS["earth"], s=60, zorder=5, label="Terre")
67.     ax_xz.set_xlabel("ΔX [km]")
68.     ax_xz.set_ylabel("ΔZ [km]")
69.     ax_xz.set_title("Plan XZ (hors-plan)")
70.     ax_xz.set_aspect("equal")
71.     ax_xz.grid(True)
72.
73.     # --- YZ ---
74.     ax_yz = fig.add_subplot(2, 2, 4)
75.     ax_yz.axis("equal")
76.     ax_yz.scatter(dpos[:, 1], dpos[:, 2], c=t_norm, cmap="plasma", s=0.5)
77.     ax_yz.scatter(0, 0, color=COLORS["L2"], s=60, zorder=5)
78.     ax_yz.scatter(0, 0, color=COLORS["earth"], s=60, zorder=5, label="Terre")
79.     ax_yz.set_xlabel("ΔY [km]")
80.     ax_yz.set_ylabel("ΔZ [km]")
81.     ax_yz.set_title("Plan YZ")
82.     ax_yz.set_aspect("equal")
83.     ax_yz.grid(True)
```

Annexe – trajectory.py

```
83. plt.tight_layout()
84. if save_path:
85.     plt.savefig(save_path, dpi=150, bbox_inches="tight")
86. plt.show()
87.
88. # Inertiel
89. def plot_inertial_trajectory(sim: InertialSimulation, save_path: str | None = None):
90.     """
91.     Trajectoire JWST + orbite Terre dans le référentiel inertiel [UA].
92.
93.     Parameters
94.     -----
95.     sim : InertialSimulation (après run())
96.     """
97.     assert sim.times is not None
98.
99.     set_style()
100.    UA = 1.495_978_707e11 # m → UA
101.
102.    pos_jwst = sim.positions / UA
103.    pos_earth = sim.earth_positions() / UA
104.    t_norm = sim.times / sim.times[-1]
105.
106.    fig = plt.figure(figsize=(14, 10))
107.    fig.suptitle("JWST — Trajectoire dans le référentiel inertiel J2000", y=0.98)
108.
109.    # --- 3D ---
110.    ax3 = fig.add_subplot(2, 2, 1, projection="3d")
111.    ax3.set_facecolor(COLORS["bg"])
112.    ax3.scatter(
113.        pos_jwst[:, 0],
114.        pos_jwst[:, 1],
115.        pos_jwst[:, 2],
116.        c=t_norm,
117.        cmap="plasma",
118.        s=0.5,
119.        alpha=0.8,
120.        label="JWST",
121.    )
```

```
122.    ax3.plot(
123.        pos_earth[:, 0],
124.        pos_earth[:, 1],
125.        pos_earth[:, 2],
126.        color=COLORS["earth"],
127.        lw=0.8,
128.        alpha=0.6,
129.        label="Terre",
130.    )
131.    # ax3.scatter(0, 0, 0, color=COLORS["sun"], s=80, zorder=5, label="Soleil")
132.    ax3.set_xlabel("X [UA]")
133.    ax3.set_ylabel("Y [UA]")
134.    ax3.set_zlabel("Z [UA]")
135.    ax3.set_title("Vue 3D")
136.    ax3.legend(fontsize=8)
137.
138.    # --- XY ---
139.    ax_xy = fig.add_subplot(2, 2, 2)
140.    sc = ax_xy.scatter(
141.        pos_jwst[:, 0], pos_jwst[:, 1], c=t_norm, cmap="plasma", s=0.5, label="JWST"
142.    )
143.    ax_xy.plot(
144.        pos_earth[:, 0],
145.        pos_earth[:, 1],
146.        color=COLORS["earth"],
147.        lw=0.8,
148.        alpha=0.6,
149.        label="Terre",
150.    )
151.    # ax_xy.scatter(0, 0, color=COLORS["sun"], s=80, zorder=5, label="Soleil")
152.    ax_xy.set_xlabel("X [UA]")
153.    ax_xy.set_ylabel("Y [UA]")
154.    ax_xy.set_title("Plan XY (écliptique)")
155.    ax_xy.set_aspect("equal")
156.    ax_xy.grid(True)
157.    ax_xy.legend(fontsize=8)
158.    add_colorbar_time(fig, ax_xy, sc, label=f"Temps [0 → {sim.times_days[-1]:.0f}]")
```

Annexe – trajectory.py

```
159. # --- Zoom autour de L2 (XY) ---
160. ax_zoom = fig.add_subplot(2, 2, 3)
161. ax_zoom.scatter(pos_jwst[:, 0], pos_jwst[:, 1], c=t_norm, cmap="plasma", s=0.5)
162. # Centre approximatif de L2 (≈ Terre + 0.01 UA dans la direction radiale)
163. ax_zoom.plot(
164.     pos_earth[:, 0], pos_earth[:, 1], color=COLORS["earth"], lw=0.8, alpha=0.4
165. )
166. ax_zoom.set_xlabel("X [UA]")
167. ax_zoom.set_ylabel("Y [UA]")
168. ax_zoom.set_title("Zoom région L2")
169. ax_zoom.grid(True)
170. # Centrage automatique sur le barycentre de la trajectoire JWST
171. cx, cy = pos_jwst[:, 0].mean(), pos_jwst[:, 1].mean()
172. half = 0.025
173. ax_zoom.set_xlim(cx - half, cx + half)
174. ax_zoom.set_ylim(cy - half, cy + half)
175. ax_zoom.set_aspect("equal")
176.
177. # --- XZ ---
178. ax_xz = fig.add_subplot(2, 2, 4)
179. ax_xz.axis("equal")
180. ax_xz.scatter(pos_jwst[:, 0], pos_jwst[:, 2], c=t_norm, cmap="plasma", s=0.5)
181. ax_xz.set_xlabel("X [UA]")
182. ax_xz.set_ylabel("Z [UA]")
183. ax_xz.set_title("Plan XZ (inclinaison)")
184. ax_xz.grid(True)
185. ax_xz.set_aspect("equal")
186.
187. plt.tight_layout()
188. if save_path:
189.     plt.savefig(save_path, dpi=150, bbox_inches="tight")
190. plt.show()
```

Annexe – velocity.py

```
83. plt.tight_layout()
84. if save_path:
85.     plt.savefig(save_path, dpi=150, bbox_inches="tight")
86. plt.show()
87.
88. # Inertiel
89. def plot_inertial_trajectory(sim: InertialSimulation, save_path: str | None = None):
90.     """
91.     Trajectoire JWST + orbite Terre dans le référentiel inertiel [UA].
92.
93.     Parameters
94.     -----
95.     sim : InertialSimulation (après run())
96.     """
97.     assert sim.times is not None
98.
99.     set_style()
100.    UA = 1.495_978_707e11 # m → UA
101.
102.    pos_jwst = sim.positions / UA
103.    pos_earth = sim.earth_positions() / UA
104.    t_norm = sim.times / sim.times[-1]
105.
106.    fig = plt.figure(figsize=(14, 10))
107.    fig.suptitle("JWST — Trajectoire dans le référentiel inertiel J2000", y=0.98)
108.
109.    # --- 3D ---
110.    ax3 = fig.add_subplot(2, 2, 1, projection="3d")
111.    ax3.set_facecolor(COLORS["bg"])
112.    ax3.scatter(
113.        pos_jwst[:, 0],
114.        pos_jwst[:, 1],
115.        pos_jwst[:, 2],
116.        c=t_norm,
117.        cmap="plasma",
118.        s=0.5,
119.        alpha=0.8,
120.        label="JWST",
121.    )
```

```
122. ax3.plot(
123.     pos_earth[:, 0],
124.     pos_earth[:, 1],
125.     pos_earth[:, 2],
126.     color=COLORS["earth"],
127.     lw=0.8,
128.     alpha=0.6,
129.     label="Terre",
130. )
131. # ax3.scatter(0, 0, 0, color=COLORS["sun"], s=80, zorder=5, label="Soleil")
132. ax3.set_xlabel("X [UA]")
133. ax3.set_ylabel("Y [UA]")
134. ax3.set_zlabel("Z [UA]")
135. ax3.set_title("Vue 3D")
136. ax3.legend(fontsize=8)
137.
138. # --- XY ---
139. ax_xy = fig.add_subplot(2, 2, 2)
140. sc = ax_xy.scatter(
141.     pos_jwst[:, 0], pos_jwst[:, 1], c=t_norm, cmap="plasma", s=0.5, label="JWST"
142. )
143. ax_xy.plot(
144.     pos_earth[:, 0],
145.     pos_earth[:, 1],
146.     color=COLORS["earth"],
147.     lw=0.8,
148.     alpha=0.6,
149.     label="Terre",
150. )
151. # ax_xy.scatter(0, 0, color=COLORS["sun"], s=80, zorder=5, label="Soleil")
152. ax_xy.set_xlabel("X [UA]")
153. ax_xy.set_ylabel("Y [UA]")
154. ax_xy.set_title("Plan XY (écliptique)")
155. ax_xy.set_aspect("equal")
156. ax_xy.grid(True)
157. ax_xy.legend(fontsize=8)
158. add_colorbar_time(fig, ax_xy, sc, label=f"Temps [0 → {sim.times_days[-1]:.0f}]")
```

Annexe – energy.py

```
1. import matplotlib.pyplot as plt
2. import numpy as np
3.
4. from .utils import COLORS, annotate_extrema, relative_drift, set_style
5.
6. def plot_jacobi(sim, save_path: str | None = None):
7.     """
8.     Constante de Jacobi au cours du temps (CR3BP).
9.
10.    Parameters
11.    -----
12.    sim : CR3BPSimulation (après run())
13.    """
14.    set_style()
15.    t = sim.times_days
16.    C = sim.jacobi
17.    drift = relative_drift(C)
18.
19.    fig, axes = plt.subplots(
20.        2, 1, figsize=(10, 7), gridspec_kw={"height_ratios": [3, 1]}
21.    )
22.    fig.suptitle("CR3BP — Conservation de la constante de Jacobi", y=0.98)
23.
24.    # Valeur absolue
25.    ax = axes[0]
26.    ax.plot(t, C, color=COLORS["jacobi"], lw=1.0)
27.    ax.axhline(
28.        C[0], color=COLORS["text"], lw=0.6, ls="--", alpha=0.5, label=f"C0 = {C[0]:.6f}"
29.    )
30.    ax.set_ylabel("C (constante de Jacobi)")
31.    ax.set_title(f"Dérive relative : {100*drift:.4f} %")
32.    ax.grid(True)
33.    ax.legend()
34.    annotate_extrema(ax, t, C)
35.
36.    # Dérive absolue
37.    ax2 = axes[1]
38.    ax2.plot(t, C - C[0], color=COLORS["jacobi"], lw=0.9)
39.    ax2.axhline(0, color=COLORS["text"], lw=0.5, ls="--", alpha=0.4)
```

```
40.    ax2.set_xlabel("Temps [jours]")
41.    ax2.set_ylabel("ΔC = C(t) - C0")
42.    ax2.grid(True)
43.    plt.tight_layout()
44.    if save_path:
45.        plt.savefig(save_path, dpi=150, bbox_inches="tight")
46.    plt.show()
47.
48. def plot_energy(sim, save_path: str | None = None):
49.     """
50.     Énergie mécanique spécifique au cours du temps (référentiel inertiel).
51.
52.    Parameters
53.    -----
54.    sim : InertialSimulation (après run())
55.    """
56.    set_style()
57.    t = sim.times_days
58.    E = sim.energy
59.    drift = relative_drift(E)
60.
61.    fig, axes = plt.subplots(
62.        2, 1, figsize=(10, 7), gridspec_kw={"height_ratios": [3, 1]}
63.    )
64.    fig.suptitle("Inertiel — Conservation de l'énergie mécanique spécifique", y=0.98)
65.
66.    ax = axes[0]
67.    ax.plot(t, E, color=COLORS["energy"], lw=1.0)
68.    ax.axhline(
69.        E[0],
70.        color=COLORS["text"],
71.        lw=0.6,
72.        ls="--",
73.        alpha=0.5,
74.        label=f"E0 = {E[0]:.4e} J/kg",
75.    )
76.    ax.set_ylabel("E spécifique [J/kg]")
77.    ax.set_title(f"Dérive relative : {100*drift:.4f} %")
78.    ax.grid(True)
79.    ax.legend()
80.    annotate_extrema(ax, t, E)
```

Annexe – energy.py

```
81. ax2 = axes[1]
82. ax2.plot(t, E - E[0], color=COLORS["energy"], lw=0.9)
83. ax2.axhline(0, color=COLORS["text"], lw=0.5, ls="--", alpha=0.4)
84. ax2.set_xlabel("Temps [jours]")
85. ax2.set_ylabel(" $\Delta E = E(t) - E_0$  [J/kg]")
86. ax2.grid(True)
87.
88. plt.tight_layout()
89. if save_path:
90.     plt.savefig(save_path, dpi=150, bbox_inches="tight")
91. plt.show()
92.
93. def plot_energy_comparison(sim_cr3bp, sim_inertial, save_path: str | None = None):
94.     """
95.     Comparaison des dérivées de conservation entre les deux simulations.
96.     Affiche  $\Delta C/C_0$  et  $\Delta E/E_0$  sur le même graphe (en %).
97.     """
98.     set_style()
99.     fig, ax = plt.subplots(figsize=(10, 5))
100.    fig.suptitle("Comparaison des dérivées numériques CR3BP vs Inertiel")
101.
102.    C = sim_cr3bp.jacobi
103.    E = sim_inertial.energy
104.    t_cr3 = sim_cr3bp.times_days
105.    t_in = sim_inertial.times_days
106.
107.    ax.plot(
108.        t_cr3,
109.        100 * (C - C[0]) / abs(C[0]),
110.        color=COLORS["jacobi"],
111.        label=" $\Delta C/C_0$  (CR3BP)",
112.        lw=1.0,
113.    )
114.    ax.plot(
115.        t_in,
116.        100 * (E - E[0]) / abs(E[0]),
117.        color=COLORS["energy"],
118.        label=" $\Delta E/E_0$  (Inertiel)",
119.        lw=1.0,
120.    )
121.
122.    ax.axhline(0, color=COLORS["text"], lw=0.5, ls="--", alpha=0.4)
123.    ax.set_xlabel("Temps [jours]")
124.    ax.set_ylabel("Dérive relative [%]")
125.    ax.grid(True)
126.    ax.legend()
127.
128.    plt.tight_layout()
129.    if save_path:
130.        plt.savefig(save_path, dpi=150, bbox_inches="tight")
131.    plt.show()
```

Annexe – stationkeeping.py

```
1. import matplotlib.pyplot as plt
2. import numpy as np
3. from cr3bp.lagrange import lagrange_L2
4. from cr3bp.stationkeeping import StationKeepingSimulation
5.
6. from .utils import COLORS, annotate_extrema, set_style
7.
8. def plot_sk_trajectory(sim_sk: StationKeepingSimulation, save_path: str | None = None):
9.     """
10.     Trajectoires comparées : avec corrections / sans corrections / référence.
11.     Vue 3D + projections 2D dans le repère tournant centré sur L2.
12.     """
13.     assert sim_sk.times is not None
14.
15.     set_style()
16.     KM = 1.496e8 # adim → km
17.     L2 = lagrange_L2(sim_sk.mu)
18.
19.     pos_sk = (sim_sk.positions - L2) * KM
20.     pos_free = (sim_sk.positions_free - L2) * KM
21.     pos_ref = (sim_sk.positions_ref - L2) * KM
22.     t_norm = sim_sk.times / sim_sk.times[-1]
23.
24.     v_s = np.asarray(sim_sk.v_s_list)[:,:3]
25.     v_s = v_s / np.linalg.norm(v_s, axis=1, keepdims=True)
26.
27.     step = max(len(pos_sk) // 200, 1)
28.     idx = np.arange(0, len(pos_sk), step, dtype=np.int64)
29.
30.     # Positions des manœuvres
31.     man_pos = (
32.         np.array(
33.             [(sim_sk._ref_state(m.t_adim)[:3] - L2) * KM for m in sim_sk.maneuvers],
34.             dtype=np.float64,
35.         )
36.         if sim_sk.maneuvers
37.         else None
38.     )
39.
40.     fig = plt.figure(figsize=(14, 10))
41.     fig.suptitle("JWST — Station-Keeping EVSK (repère tournant, centré sur L2)")
42.
43.     # --- 3D ---
44.     ax3 = fig.add_subplot(2, 2, 1, projection="3d")
45.     ax3.set_facecolor(COLORS["bg"])
46.     ax3.plot(
47.         pos_ref[:, 0],
48.         pos_ref[:, 1],
49.         pos_ref[:, 2],
50.         color=COLORS["L2"],
51.         lw=0.8,
52.         alpha=0.5,
53.         label="Référence",
54.     )
55.     ax3.scatter(
56.         pos_sk[:, 0],
57.         pos_sk[:, 1],
58.         pos_sk[:, 2],
59.         c=t_norm,
60.         cmap="plasma",
61.         s=0.4,
62.         label="Avec SK",
63.     )
64.     ax3.plot(
65.         pos_free[:, 0],
66.         pos_free[:, 1],
67.         pos_free[:, 2],
68.         color="#888888",
69.         lw=0.5,
70.         alpha=0.4,
71.         label="Sans SK",
72.     )
73.     if man_pos is not None:
74.         ax3.scatter(
75.             man_pos[:, 0],
76.             man_pos[:, 1],
77.             man_pos[:, 2],
78.             color="white",
79.             s=20,
80.             zorder=6,
81.             marker="x",
82.             label="Manœuvre",
83.         )
```

Annexe – stationkeeping.py

```
84. ax3.set_xlabel("ΔX [km]")
85. ax3.set_ylabel("ΔY [km]")
86. ax3.set_zlabel("ΔZ [km]")
87. ax3.set_title("Vue 3D")
88. ax3.legend(fontsize=7)
89.
90. # --- XY ---
91. ax_xy = fig.add_subplot(2, 2, 2)
92. ax_xy.plot(pos_ref[:, 0], pos_ref[:, 1], color=COLORS["L2"], lw=0.8, alpha=0.5)
93. ax_xy.scatter(pos_sk[:, 0], pos_sk[:, 1], c=t_norm, cmap="plasma", s=0.4)
94. ax_xy.plot(pos_free[:, 0], pos_free[:, 1], color="#888888", lw=0.5, alpha=0.4)
95. if man_pos is not None:
96.     ax_xy.scatter(
97.         man_pos[:, 0], man_pos[:, 1], color="white", s=20, marker="x", zorder=6
98.     )
99. ax_xy.set_xlabel("ΔX [km]")
100. ax_xy.set_ylabel("ΔY [km]")
101. ax_xy.set_title("Plan XY")
102. ax_xy.grid(True)
103. ax_xy.set_aspect("equal")
104.
105. # --- Erreur de position ---
106. ax_err = fig.add_subplot(2, 2, 3)
107. err_sk = sim_sk.position_errors
108. err_free = np.linalg.norm(pos_free - pos_ref, axis=1)
109. t_days = sim_sk.times_days
110. ax_err.semilogy(t_days, err_sk, color=COLORS["jwst"], lw=1.0, label="Avec SK")
111. ax_err.semilogy(
112.     t_days, err_free, color="#888888", lw=0.8, alpha=0.7, label="Sans SK"
113. )
114. if sim_sk.maneuvers:
115.     for m in sim_sk.maneuvers:
116.         ax_err.axvline(m.t_days, color="white", lw=0.4, alpha=0.3)
117. ax_err.set_xlabel("Temps [jours]")
118. ax_err.set_ylabel("Erreur position [km]")
119. ax_err.set_title("Erreur par rapport à la référence")
120. ax_err.grid(True)
121. ax_err.legend()
```

```
122. # --- YZ ---
123. ax_yz = fig.add_subplot(2, 2, 4)
124. ax_yz.axis("equal")
125. ax_yz.plot(pos_ref[:, 1], pos_ref[:, 2], color=COLORS["L2"], lw=0.8, alpha=0.5)
126. ax_yz.scatter(pos_sk[:, 1], pos_sk[:, 2], c=t_norm, cmap="plasma", s=0.4)
127. ax_yz.plot(pos_free[:, 1], pos_free[:, 2], color="#888888", lw=0.5, alpha=0.4)
128. ax_yz.set_xlabel("ΔY [km]")
129. ax_yz.set_ylabel("ΔZ [km]")
130. ax_yz.set_title("Plan YZ")
131. ax_yz.grid(True)
132. ax_yz.set_aspect("equal")
133.
134. scale = 200e3 # ajuster visuellement
135.
136. for m, v in zip(sim_sk.maneuvers, sim_sk.v_s_list):
137.     # retrouver l'indice temporel correspondant
138.     i = np.argmin(np.abs(sim_sk.times - m.t_adim))
139.
140.     v = v[:3]
141.     n = np.linalg.norm(v)
142.     if n > 0:
143.         v = v / n
144.
145.     ax3.quiver(
146.         pos_sk[i, 0],
147.         pos_sk[i, 1],
148.         pos_sk[i, 2],
149.         v[0],
150.         v[1],
151.         v[2],
152.         length=int(scale),
153.         color="red",
154.         alpha=0.8,
155.         linewidth=1.0,
156.     )
```

Annexe – stationkeeping.py

```
157. ax_xy.quiver(
158.     pos_sk[i, 0],
159.     pos_sk[i, 1],
160.     v[0],
161.     v[1],
162.     color="red",
163.     alpha=0.8,
164.     linewidth=1.0,
165. )
166.
167. ax_yz.quiver(
168.     pos_sk[i, 1],
169.     pos_sk[i, 2],
170.     v[1],
171.     v[2],
172.     color="red",
173.     alpha=0.8,
174.     linewidth=0.4,
175. )
176.
177. plt.tight_layout()
178. if save_path:
179.     plt.savefig(save_path, dpi=150, bbox_inches="tight")
180. plt.show()
181.
182. def plot_delta_v_history(
183.     sim_sk: StationKeepingSimulation, save_path: str | None = None
184. ):
185.     """
186.     Historique des ΔV : magnitude par manœuvre et cumulé.
187.     """
188.     set_style()
189.     if not sim_sk.maneuvers:
190.         print("Aucune manœuvre enregistrée.")
191.         return
192.
193.     t_man = sim_sk.maneuver_times_days
194.     dvs = sim_sk.dv_norms_ms
195.     cumul = np.cumsum(dvs)
196.
197.     fig, axes = plt.subplots(2, 1, figsize=(10, 7))
198.     fig.suptitle("EVSK — Historique des manœuvres de station-keeping")
199.
200.     ax = axes[0]
201.     ax.bar(
202.         t_man,
203.         dvs,
204.         width=1.5,
205.         color=COLORS["speed"],
206.         alpha=0.8,
207.         label="|ΔV| par manœuvre",
208.     )
209.     ax.axhline(
210.         dvs.mean(),
211.         color=COLORS["text"],
212.         lw=0.8,
213.         ls="--",
214.         label=f"Moyenne = {dvs.mean():.4f} m/s",
215.     )
216.     ax.set_ylabel("|ΔV| [m/s]")
217.     ax.set_title(
218.         f"Budget total : {sim_sk.total_dv_ms:.3f} m/s " f"(JWST réel : ~2.5 m/s/an)"
219.     )
220.     ax.grid(True)
221.     ax.legend()
222.     annotate_extrema(ax, t_man, dvs, n=1)
223.
224.     ax2 = axes[1]
225.     ax2.plot(t_man, cumul, color=COLORS["energy"], lw=1.2, marker="o", ms=3)
226.     ax2.set_xlabel("Temps [jours]")
227.     ax2.set_ylabel("ΔV cumulé [m/s]")
228.     ax2.set_title("Budget ΔV cumulé")
229.     ax2.grid(True)
230.
231.     plt.tight_layout()
232.     if save_path:
233.         plt.savefig(save_path, dpi=150, bbox_inches="tight")
234.         plt.show()
235.
236. def plot_sk_jacobi(sim_sk: StationKeepingSimulation, save_path: str | None = None):
237.     """
238.     Constante de Jacobi avec les instants de manœuvre marqués.
239.     Les manœuvres brisent la conservation de C ( $\Delta C = \Delta V \cdot dC/dv \neq 0$  en général).
240.     """
```

Annexe – stationkeeping.py

```
239. assert sim_sk.jacobi is not None
240. set_style()
241. t = sim_sk.times_days
242. C = sim_sk.jacobi
243.
244. fig, ax = plt.subplots(figsize=(10, 4))
245. fig.suptitle("Station-Keeping — Constante de Jacobi (avec sauts aux manœuvres)")
246.
247. ax.plot(t, C, color=COLORS["jacobi"], lw=0.8)
248. ax.axhline(
249.     C[0], color=COLORS["text"], lw=0.5, ls="--", alpha=0.4, label=f"C0 = {C[0]:.6f}"
250. )
251. for m in sim_sk.maneuvers:
252.     ax.axvline(m.t_days, color=COLORS["speed"], lw=0.5, alpha=0.5)
253. ax.set_xlabel("Temps [jours]")
254. ax.set_ylabel("C (constante de Jacobi)")
255. ax.set_title("Les traits verticaux indiquent les manœuvres")
256. ax.grid(True)
257. ax.legend()
258.
259. plt.tight_layout()
260. if save_path:
261.     plt.savefig(save_path, dpi=150, bbox_inches="tight")
262. plt.show()
```